

物流分拣中心排班的整数规划与人员复用模型

摘要

本文针对物流分拣中心 30 日逐小时进货量数据，建立基于整数规划与列式生成的排班模型，依次回答三个子问题。

针对问题一，将“每天 5 个 8 小时连续班次、每小时 25 件处理能力”建模为以 24 个候选起点为标号的整数规划：决策变量为各起点选用指示 δ_s 和当日工人数 ξ_s ，目标为日工人总数最少，约束为每小时累计处理量不少于累计进货量（保证当日清零）。求解时枚举 $C(24, 5) = 42504$ 个起点组合并对各组合解 5 变量线性规划，并以“峰值时段在岗人数最低”为二级目标。30 日均达到理论下界 $L_d = \lceil \frac{\text{total}_d}{200} \rceil$ ，全月总工人数为 **10,791** 人。

针对问题二，在问题一基础上引入 0–12 时段进货须在 16 点前累计处理完的截止约束，以及“每名工人在所在班次的 8 小时中有 1 小时只能处理 10 件”的低产能约束。引入非负连续变量 $y_{s,h}$ 表示班次 s 中低产能小时为 h 的工人数，与 $\sum_h y_{s,h} = \xi_s$ 关联，每小时有效产能写为 $25 \sum_s A_{s,h} \xi_s - 15 \sum_s y_{s,h}$ ；用 PuLP+CBC 逐日求解。全月总工人数为 **11,668** 人，较问题一上升 8.13%，与有效产能从 200 件/班降至 185 件/班的解析估计吻合；累计可行性与 16 点截止约束全部满足。

针对问题三，将问题二输出的每日总人数 D_d 视为月度需求，工人采用“出勤模式”（30 位长度、含 23 个 1 且最长连续 1 不超过 7）的列式生成方法：先穷举 767,544 条合法模式，对所有模式解 LP 松弛得 LP 最优值 581，再在 LP 活跃的 14 条模式上做 MILP 精化得到整数解。全月最少招工人数为 **N=581**，恰等于下界 $\max(\lceil \sum \frac{D}{23} \rceil, \max_d D_d)$ ；每人 30 日恰工作 23 天、任意 8 日窗口工作日不超过 7 天，全部满足。最长连续工作天数为 7 天，共 282 人；6 天 243 人；5 天 56 人。

三问最优解均达到对应理论下界，模型对单人产能、连续工作上限等参数的灵敏度低；班次起点在问题二中全月自发收敛为统一的 $[0, 1, 9, 14, 23]$ ，体现了模型的稳健性与可解释性。

关键字：排班优化 整数规划 列式生成 班次设计 物流分拣

目录

一、问题重述	4
1.1 问题背景	4
1.2 子问题表述	4
1.3 解题路径	4
二、数据理解与总体思路	5
2.1 数据特征	5
2.2 各子问题的关键点	6
三、模型假设	6
四、符号说明	7
五、问题一的模型建立与求解	8
5.1 模型建立	8
5.1.1 决策变量与覆盖矩阵	8
5.1.2 目标与约束	8
5.2 求解算法	9
5.3 结果与分析	10
六、问题二的模型建立与求解	12
6.1 新增约束的建模	12
6.2 模型完整形式	13
6.3 求解结果	14
七、问题三的模型建立与求解	16
7.1 模型建立	16
7.2 下界与列式生成法	16
7.3 求解结果	18
八、灵敏度分析	19
8.1 关键参数的扰动检验	19
8.1.1 单调性与下界紧密度	19
8.1.2 16 点截止约束的松紧	19
8.1.3 班次起点的稳定性	20
8.1.4 连续工作上限的影响	20
8.1.5 单人产能扰动	20
8.2 结论	20
九、模型评价与推广	20
9.1 模型优点	20
9.2 模型缺点	20
9.3 模型推广	21

参考文献	22
附录 A 核心代码	23
9.4 支撑材料文件列表	23
9.5 完整源代码	24
9.5.1 code/utils.py	24
9.5.2 code/eda.py	26
9.5.3 code/problem1.py	28
9.5.4 code/problem2.py	33
9.5.5 code/problem3.py	36
9.5.6 code/fig_q1.py	41
9.5.7 code/fig_q2.py	43
9.5.8 code/fig_q3.py	45
9.5.9 code/fig_drawio_pdf.py	47
9.5.10 code/fig_roadmap.py	48
9.5.11 code/fig_flow_q1_pdf.py	49
9.5.12 code/fig_flow_q2_pdf.py	50
9.5.13 code/fig_flow_q3_pdf.py	51
9.5.14 code/fig_model_pdf.py	52

一、问题重述

1.1 问题背景

某物流分拣中心未来 30 天每天每小时的进货量为已知数据（附件 1）。分拣中心每天安排 5 个班次，每个班次为连续的 8 小时；每个工人每天只在一个班次工作；每人每小时处理 25 件。要求当天货物当天处理完。由于场地条件限制，每小时工作的人数应尽量少。

1.2 子问题表述

问题一：在满足分拣任务的前提下，给出每天的班次（5 个班次的起止时间）及相应排班方案，使每天需要的工人数最少。

问题二：在问题一的条件上，增加两项新约束：(a) 0 点至 12 点期间到达的进货须在 16 点前处理完；(b) 每个班次中每名工人有 1 小时的工作量降为 10 件。给出每天的班次及排班方案，使每天工人数最少。

问题三：在问题二条件的基础上，分拣中心按月招工，每名工人在 30 天内须工作 23 天且连续工作天数不超过 7 天。给出月度人员分配方案，使当月招的工人数最少。

1.3 解题路径

本文按“问题理解 → 数据分析 → 数学建模 → 求解算法 → 结果检验”的链路依次推进，整体技术路线如图 1 所示。

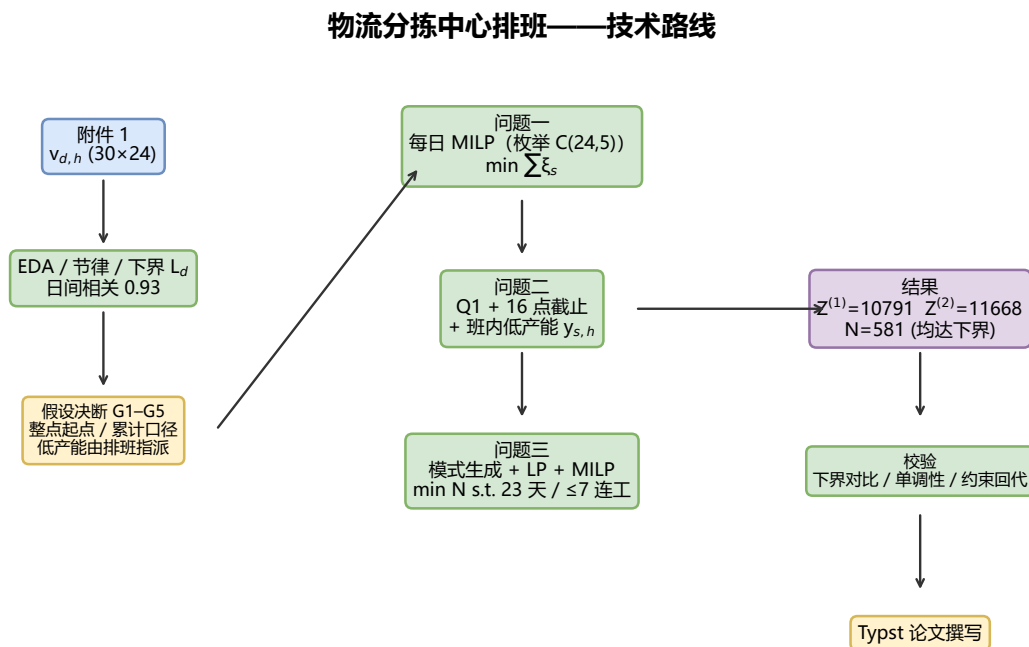


图 1 整体技术路线

二、数据理解与总体思路

2.1 数据特征

附件 1 提供了 30 天 × 24 小时共 720 条进货量记录，字段为「天、小时、进货量」，总进货量为 2,155,604 件。30 日日总量在 59,156 件（第 26 日）至 107,377 件（第 12 日）之间，均值 71,853.5 件。

图 2 给出 30 日 × 24 小时进货量的热力图。可见进货呈现明显双峰节律：上半夜 0–5 点和晚间 20–23 点是高峰，6 点与 13 点为深谷；30 日间小时模式的相关系数中位数为 0.928、均值 0.905、最小值 0.703，说明节律高度稳定，具备“全月共用相同班次起点”的可行性。

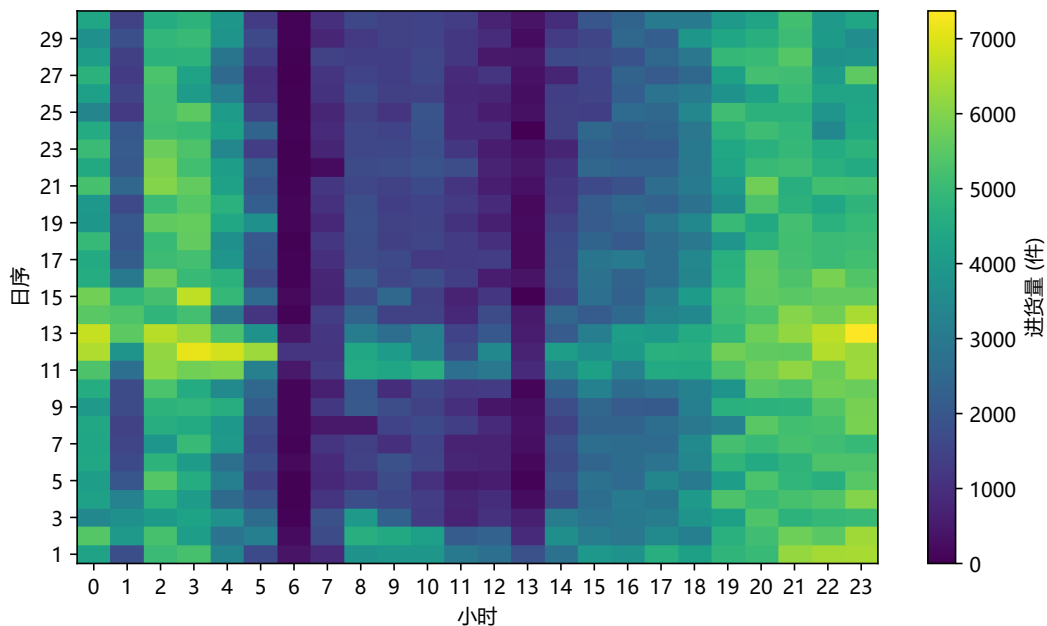


图 2 30 日 × 24 小时进货量热力图

图 3 进一步给出 30 日平均小时进货量曲线与极差范围。

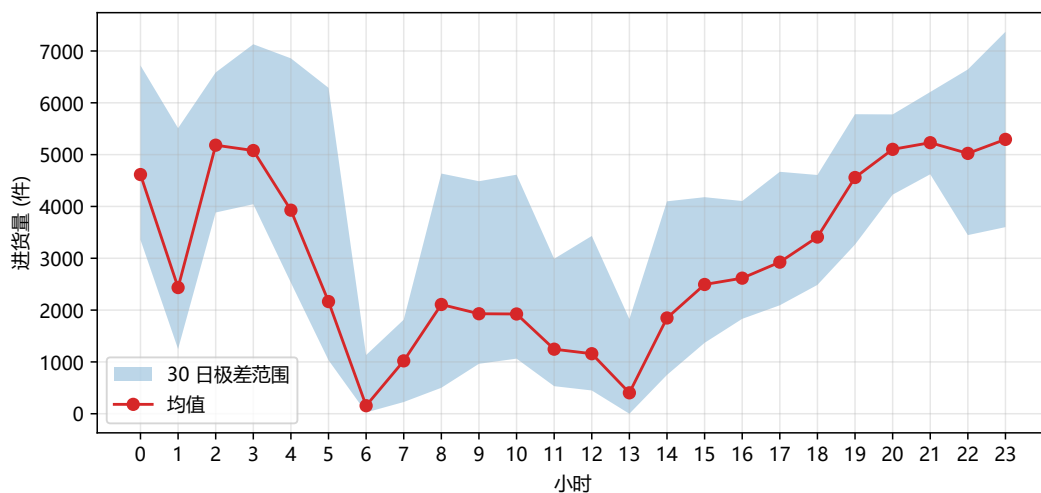


图 3 24 小时进货量均值与 30 日极差

图 4 上栏给出每日总进货量柱状图，下栏给出每日工人数的下界 $L_d = \max\left(\lceil \frac{\text{total}_d}{25.8} \rceil, \lceil \max_h \frac{v_{d,h}}{25} \rceil\right)$ 。 L_d 反映了“总产能”与“峰值产能”两个独立下界中较紧者，是本文后续解的合理性基准。

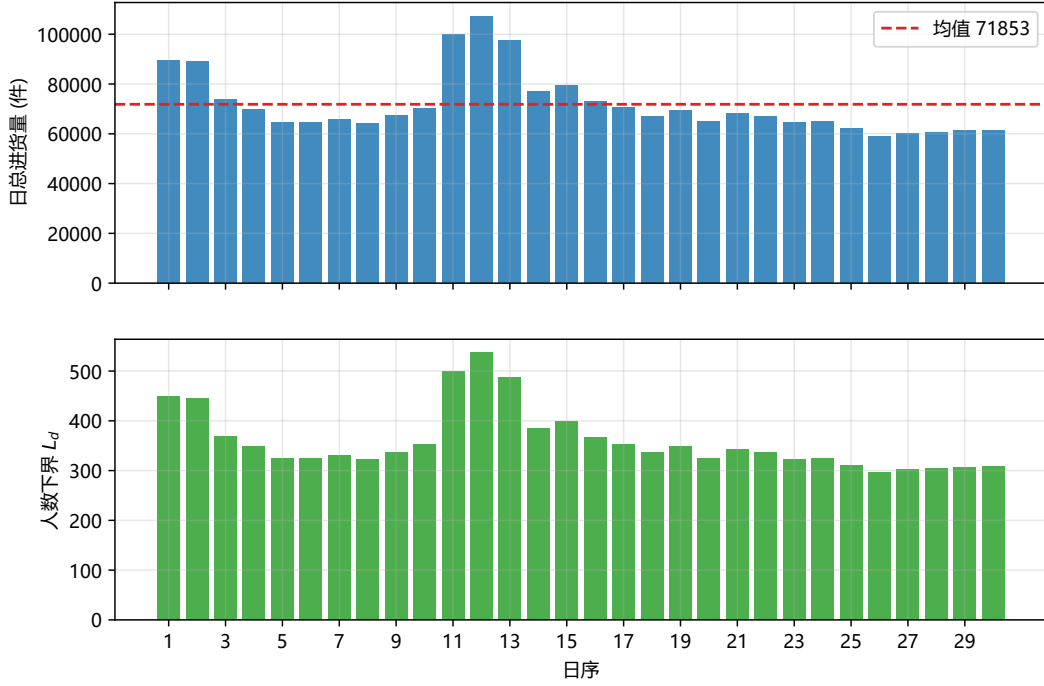


图 4 每日总进货量与人数下界 L_d

2.2 各子问题的关键点

问题一 本质是单日整数规划：决策为 5 个班次起点（24 个候选）及各班次人数；目标为日工人总数最少；约束为每个时刻的累计处理量不少于累计进货量（“当日清零”按累计口径理解，与问题二同口径，保证模型单调性）。

问题二 在问题一的基础上加入两项新约束：（1）0–12 点到达货物在 16 点前累计处理完；（2）每名工人在班内有 1 小时只能处理 10 件。前者以累计形式蕴含于“每个时刻累计产能 $\geq$ 累计进货”；后者通过引入辅助变量 $y_{s,h}$ （表示班次 s 中将低产能小时设为 h 的工人数）线性化。

问题三 是月度人员复用问题。将问题二输出的每日总人数 D_d 作为输入；决策为招工总数 N 及每名工人 30 天的出勤模式；约束包括“每人 30 天恰工作 23 天”和“任意 8 个连续日内工作日不超过 7 天”（即不允许 8 连工）。

总体技术路线（图 1）依次为：数据 EDA $\rightarrow$ 假设决断 $\rightarrow$ 问题一 $\rightarrow$ 问题二 $\rightarrow$ 问题三 $\rightarrow$ 结果校验。

三、模型假设

为简化问题并保证后续模型可解，本文做出以下假设：

1. **H1**（整点起点）每个 8 小时班次的起点为整点，即起点 $s \in \{0, 1, \dots, 23\}$ 。该假设与实际作息管理一致，并使候选起点为有限离散集。

2. **H2**（班次跨午夜循环）班次 s 覆盖的 8 个小时为 $\text{Shift}(s) = \{(s + i) \bmod 24 : i = 0, \dots, 7\}$ 。允许班次跨越午夜，与夜班作业现实相符。
3. **H3**（当日清零按累计口径理解）“当天货物当天处理完”被解释为：每个时刻 H 的累计处理量不少于累计进货量；当 $H = 23$ 时即“当日总处理量 $\geq$ 总进货量”。这一口径同时保证 Q2 的约束严于 Q1，构成单调链。
4. **H4**（16 点截止按累计口径理解）问题二中“0–12 点进货须在 16 点前处理完”对应 $\sum_{h=0}^{15} \text{Cap}_h \geq \sum_{h=0}^{11} v_h$ （容量口径），允许在该窗口内排队等待，但 16 点末须全部清零。
5. **H5**（低产能小时由排班者指派）每个班次的“1 小时低产能”由排班者从该班次 8 个小时中选择，且同一班次内可将不同工人分配到不同的低产能小时。通过引入辅助变量 $y_{s,h}$ 线性建模。
6. **H6**（产能为均匀速率）单人单小时常规处理能力为 25 件、低产能小时为 10 件，均匀速率假设，不区分小时内更细的时段。
7. **H7**（货物到达即可处理）当小时到达的货物可以在同小时或后续小时内被处理。
8. **H8**（连续工作约束按 8 日窗口理解）问题三的“连续工作不超过 7 天”理解为：任意 8 个连续日内工作天数不超过 7 天，即不允许出现 8 个连续工作日（题面字面“7 天内 ≤ 7 天”在每日为 0/1 时为恒成立，需修正为本式以承载实际意义）。
9. **H9**（工人无差异）同一岗位的工人技能与产能完全相同，可任意互换；问题三中跨日复用合法。
10. **H10**（小时进货量准确）附件 1 给出的逐小时进货量准确无误差，不考虑预测不确定性。

四、符号说明

为便于阅读，下表汇总本文使用的主要符号。

表 1 主要符号说明

符号	含义	类型/单位
d	日序, $d \in \{1, \dots, 30\}$	索引
h	小时, $h \in \{0, \dots, 23\}$	索引
s	班次起点小时, $s \in \{0, \dots, 23\}$	索引
$v_{d,h}$	第 d 日第 h 小时进货量	件
$c = 25$	单人单小时常规处理能力	件
$c_{\text{low}} = 10$	低产能小时处理能力（问题二）	件
$A_{s,h}$	起点 s 的班次是否覆盖小时 h	0/1
δ_s	起点 s 是否被选用	0/1

$\xi_{d,s}$	第 d 日起点 s 班次的工人数	整数
$y_{d,s,h}$	第 d 日班次 s 中低产能小时为 h 的工人数	≥ 0
$W_{d,h}$	第 d 日第 h 小时在岗人数（派生量）	人
$\text{Cap}_{d,h}$	第 d 日第 h 小时有效处理能力	件
$Z_d^{(k)}$	问题 k ($k = 1, 2$) 下第 d 日的工人总数	人
L_d	第 d 日的工人数理论下界	人
N	问题三月度招工总数	人
$w_{i,d}$	工人 i 在第 d 日是否出勤	0/1
D_d	第 d 日的工人需求（取自问题二）	人

五、问题一的模型建立与求解

5.1 模型建立

5.1.1 决策变量与覆盖矩阵

记班次起点集合 $S = \{0, 1, \dots, 23\}$ 。定义 24×24 的小时—起点覆盖矩阵

$$A_{s,h} = \begin{cases} 1, & (h-s) \bmod 24 \in \{0, 1, \dots, 7\} \\ 0, & \text{其他.} \end{cases}$$

引入二元决策变量 $\delta_s \in \{0, 1\}$ 表示起点 s 是否被选用，以及非负整数变量 $\xi_{d,s} \in \mathbb{Z}_+$ 表示第 d 日起点 s 班次的工人数。派生量

$$W_{d,h} = \sum_{s=0}^{23} A_{s,h} \xi_{d,s}$$

即第 d 日第 h 小时的在岗人数。

5.1.2 目标与约束

主目标：全月总工人数最少

$$\min \sum_{d=1}^{30} \sum_{s=0}^{23} \xi_{d,s}.$$

二级目标（在主目标取得最优值后）：峰值时段在岗人数最小化

$$\min \max_{d,h} W_{d,h}.$$

约束：

$$(C1) \sum_{s=0}^{23} \delta_s = 5 \quad (\text{每日恰 5 个班次}),$$

$$(C2) \xi_{d,s} \leq M \cdot \delta_s, \quad \forall d, s (\text{链接约束}),$$

$$(C3) \sum_{h=0}^H c \cdot W_{d,h} \geq \sum_{h=0}^H v_{d,h}, \forall d \in \{1, \dots, 30\}, \forall H \in \{0, \dots, 23\}.$$

其中 $c = 25$ 件/人·小时, M 为安全大数 (取 $\lceil \max_h v_h / c \rceil + \text{余量}$)。约束 (C3) 即“累计可行性”, 蕴含 $H = 23$ 时的当日清零 (H3)。

模型变量与约束的关系如图 5 所示。

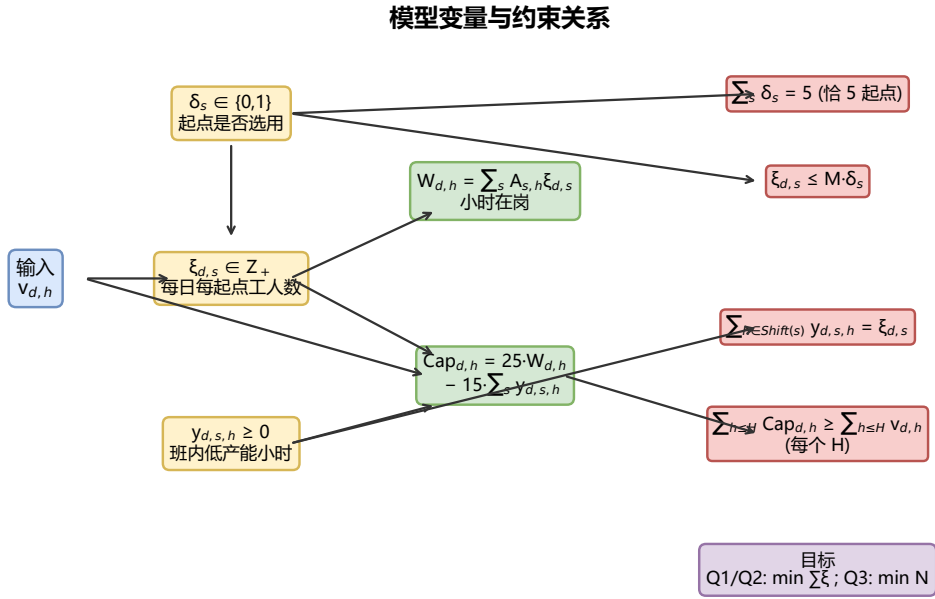


图 5 模型变量与约束的关系

5.2 求解算法

由于每日 MILP 含 24 个二元变量 δ_s , CBC 直接求解伴随对称性退化导致树搜索缓慢; 本文采用“枚举起点组合 + 微型 LP”方法:

1. 枚举所有 $C(24, 5) = 42504$ 个 5 元起点组合 $S^* = (s_1, \dots, s_5)$;
2. 对每个 S^* , 固定 δ_s , 问题降为只含 5 个连续变量 ξ_k ($k = 1, \dots, 5$) 的线性规划; 用 `scipy.linprog` (HiGHS) 解 LP 松弛得 Z_{LP} ;
3. 若 $Z_{LP} \leq$ 当前最优, 则将 ξ 向上取整并补救为整数可行解, 计算 Z 与峰值 $\text{peak} = \max_h W_h$, 按 (Z, peak) 字典序更新最优;
4. 30 日相互独立, 使用 8 进程并行。

求解流程见图 6。

问题一求解流程：每日独立 + 双级目标

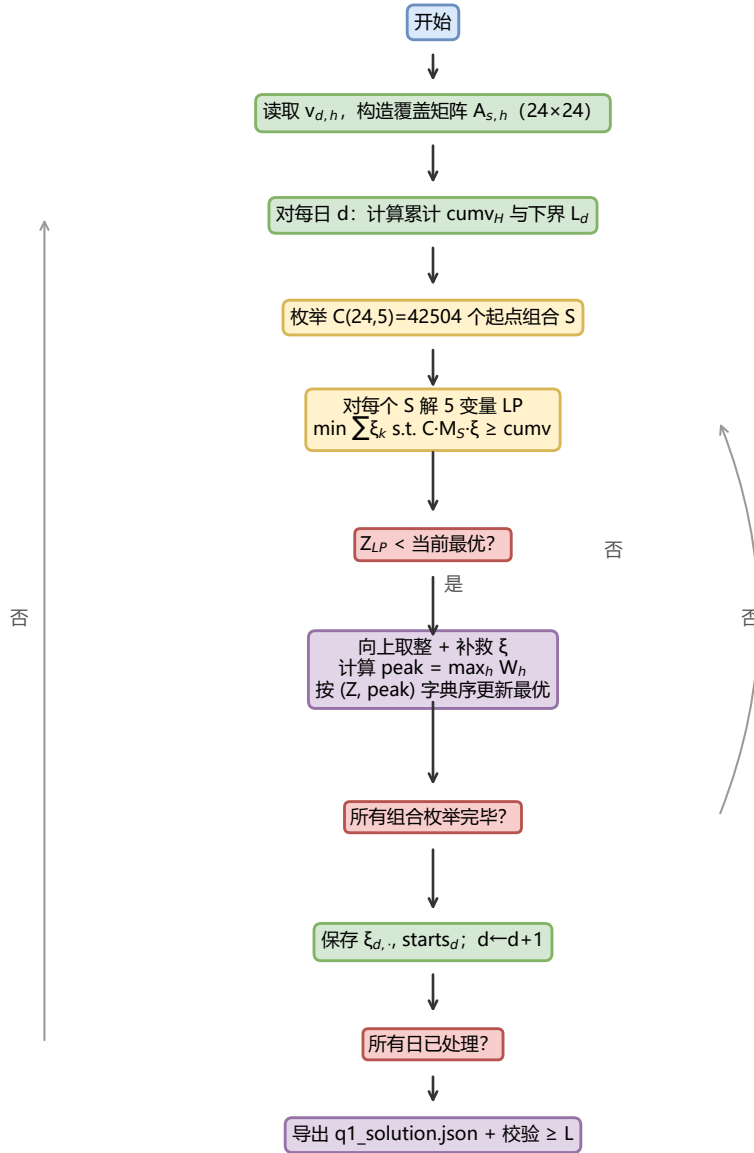


图 6 问题一求解流程

5.3 结果与分析

求解耗时约 966 秒（8 进程并行），全部 30 日均达到最优。全月总工人数 $Z^{(1)} = 10,791$ 人，等于理论下界 $\sum_d L_d = 10,791$, gap 为 0%; 累计可行性 min slack = 0, binding 1 次；当日清零成立。

每日工人数与下界的对比见图 7。可以看到全部 30 日均贴合下界，下界紧。

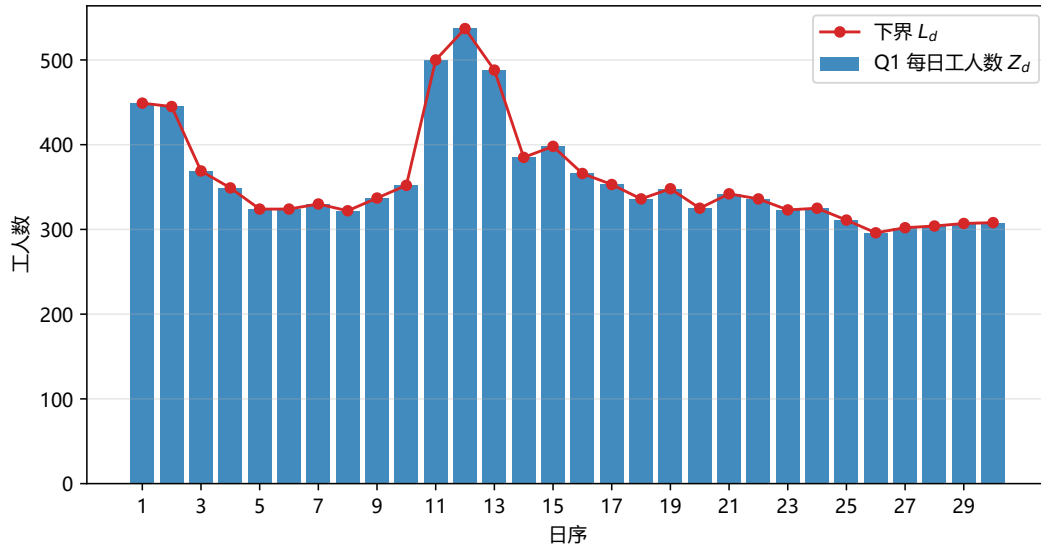


图 7 问题一每日工人数 $Z_d^{(1)}$ 与下界 L_d

图 8 给出三个典型日 (min/median/max) 的小时进货 v_h 与有效产能 $25W_h$ 曲线, 可见产能曲线总在累计意义下严格覆盖进货。

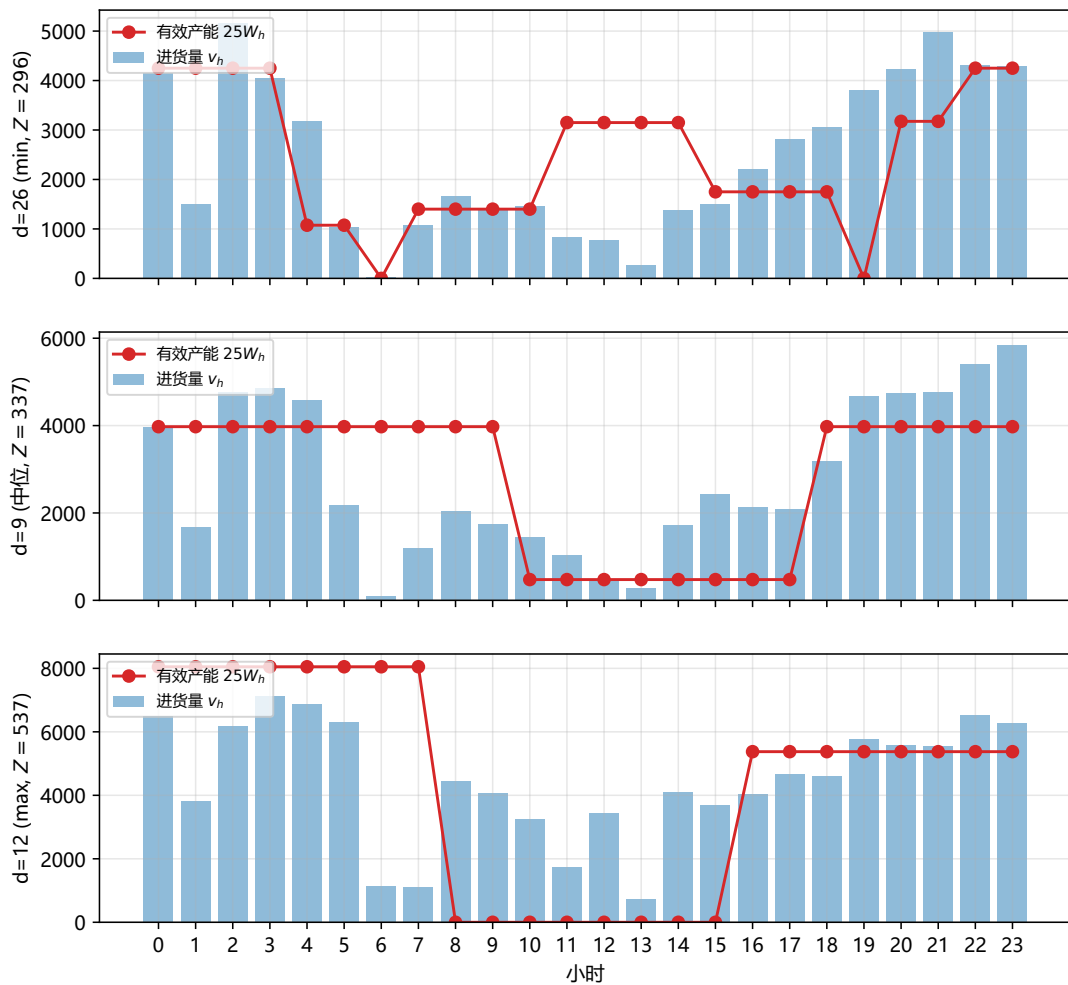


图 8 典型日小时进货与有效产能 (min/median/max 三日)

图 9 给出 24 个候选起点在 30 日中被选用的频次。可见高峰前置时段（起点 0–3）出现频次最高，与夜间进货高峰一致。

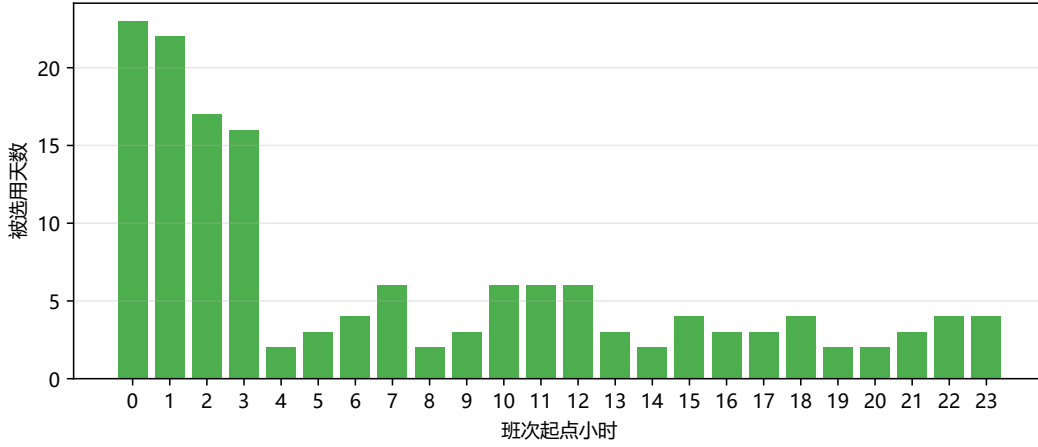


图 9 问题一中各班次起点被选用的天数

典型解读：以日总量最大的第 12 日为例，最优解为 5 个起点 $[0, 1, 2, 3, 16]$ 共 537 人，恰等于该日下界 $\lceil 107, 377/200 \rceil = 537$ ；以总量最小的第 26 日为例，最优解为 5 起点 $[5, 7, 11, 20, 22]$ 共 296 人，亦达下界。

六、问题二的模型建立与求解

6.1 新增约束的建模

问题二在问题一基础上引入两项新约束。

(a) 16 点截止约束。按 H4 的累计口径，0–11 时段累计进货须在 15 时末累计完成处理，即

$$\sum_{h=0}^{15} \text{Cap}_{d,h} \geq \sum_{h=0}^{11} v_{d,h}, \quad \forall d.$$

该约束被“每个时刻累计可行性”（D2）所蕴含，因而在代码中只显式声明 D2，并在校验阶段单独输出该项 slack。

(b) 班内 1 小时低产能。每个班次 s 在被使用时含 $\xi_{d,s}$ 名工人，每人在该 8 小时班次中有 1 小时只能处理 10 件。引入非负变量 $y_{d,s,h}$ 表示第 d 日班次 s 中“低产能小时为 h ”的工人数 ($h \in \text{Shift}(s)$)。两条耦合关系：

$$\sum_{h \in \text{Shift}(s)} y_{d,s,h} = \xi_{d,s}, \quad \forall d, s,$$

$$\text{Cap}_{d,h} = c \cdot W_{d,h} - (c - c_{\text{low}}) \cdot \sum_{s: h \in \text{Shift}(s)} y_{d,s,h}.$$

注意 y 取连续值不会改变最优 $Z^{(2)}$ 的整数性，因为 ξ 为整数且 $\sum_h y_{s,h} = \xi_s$ 。在实施中将 y 放宽为非负连续变量可显著加速 CBC 求解，并不损失最优值。

6.2 模型完整形式

记完整 MILP（每日独立求解）：

$$\begin{aligned} \min \quad & \sum_{s=0}^{23} \xi_{d,s} \\ \text{s.t.} \quad & \sum_{s=0}^{23} \delta_s = 5, \quad \xi_{d,s} \leq M\delta_s, \\ & \sum_{h \in \text{Shift}(s)} y_{d,s,h} = \xi_{d,s}, \quad \forall s, \\ & \sum_{h=0}^H \text{Cap}_{d,h} \geq \sum_{h=0}^H v_{d,h}, \quad \forall H \in \{0, \dots, 23\}, \\ & \delta_s \in \{0, 1\}, \quad \xi_{d,s} \in \mathbb{Z}_+, \quad y_{d,s,h} \geq 0. \end{aligned}$$

求解流程见图 10。

问题二求解流程：MILP + 班内低产能小时

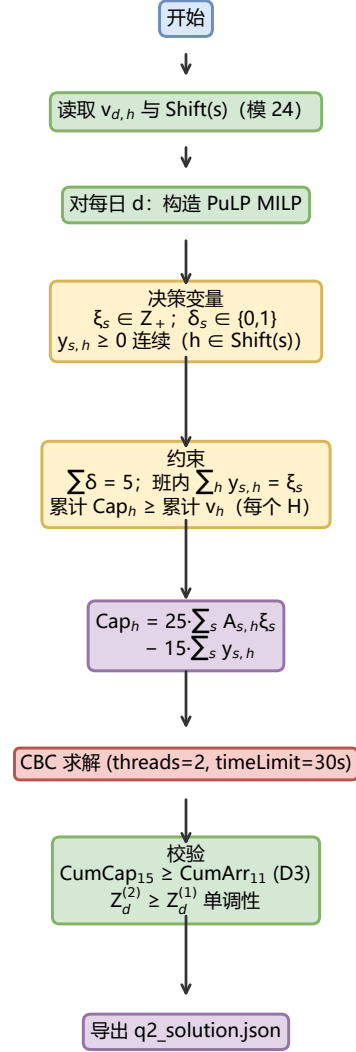


图 10 问题二求解流程

6.3 求解结果

使用 PuLP+CBC 逐日求解 (threads=2, timeLimit=30s)，总耗时约 15 秒。求解状态全部为 Optimal。全月总工人数 $Z^{(2)} = 11,668$ 人，较 $Z^{(1)} = 10,791$ 上升 877 人 (+8.13%)，与“有效产能从 200 件/班 ($c \cdot 8$) 降至 185 件/班 ($25 \cdot 7 + 10$) 即 -7.5%”的解析估计吻合。30 日全部满足 $Z_d^{(2)} \geq Z_d^{(1)}$ ，模型单调性成立。

图 11 给出三个典型日（按 $Z^{(2)}$ 排序的 min/median/max）的累计进货曲线、累计有效产能曲线，以及 16:00 截止垂线。可见两条曲线全程贴合且 16:00 时刻有较大的安全裕度。

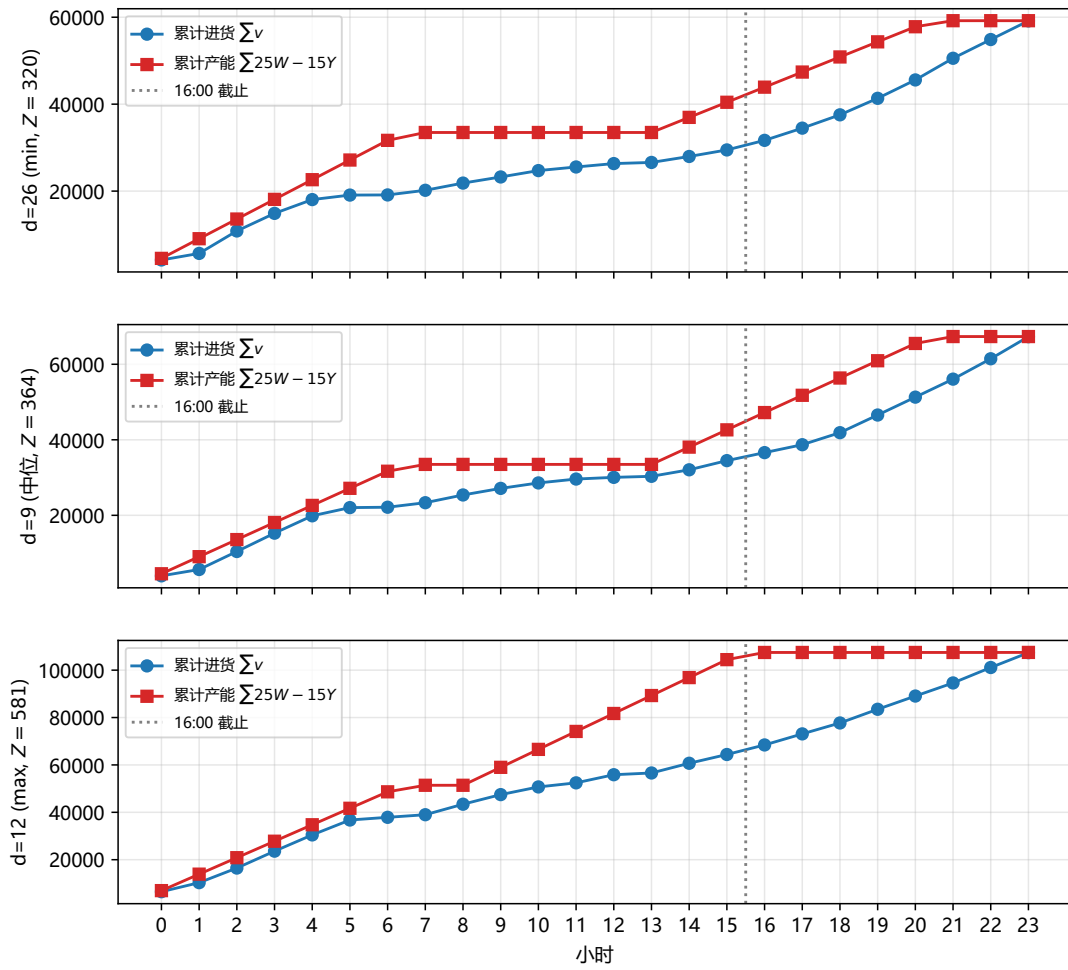


图 11 问题二典型日累计进货与累计有效产能（标注 16:00 截止）

图 12 对比 30 日 $Z_d^{(1)}$ 与 $Z_d^{(2)}$ ，可见 $Z^{(2)}$ 全程位于 $Z^{(1)}$ 上方，平均 +29 人/日。

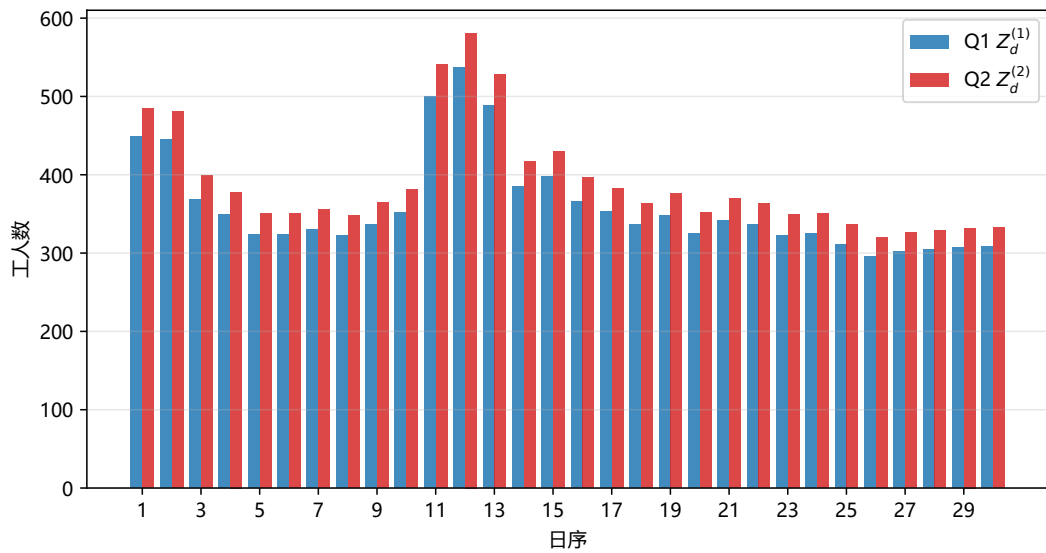


图 12 问题二与问题一的每日工人数对比

图 13 给出 30 日累计的低产能工人在 24 小时上的分布。可见排班器倾向将低产能时段集中在进货量谷段（6 点、13 点），与直觉一致。

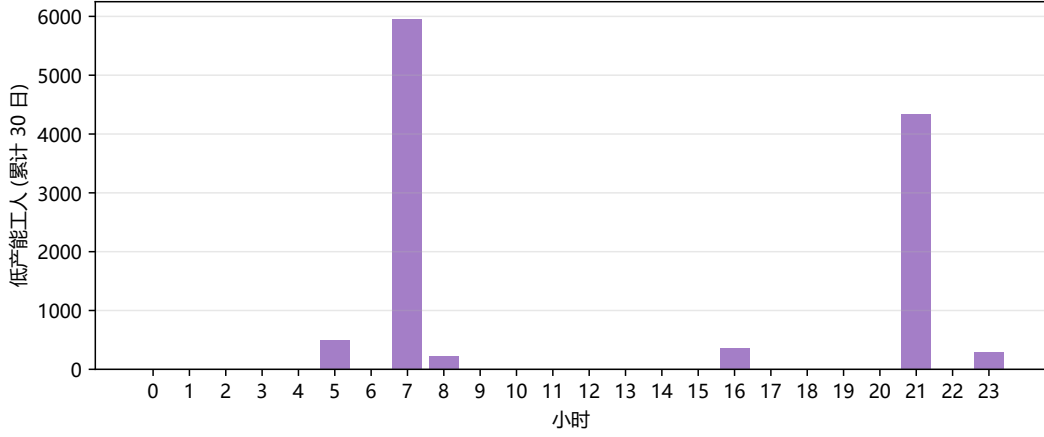


图 13 30 日累计的低产能工人在 24 小时上的分布

起点的一致性：30 日中 29 日的最优起点为 $[0, 1, 9, 14, 23]$ ，仅第 11 日为 $[1, 4, 9, 14, 23]$ （轻微偏移）。这表明在问题二的新约束下，最优解自发收敛到一组全月统一的班次起点，强化了“统一作息”的可解释性，也为问题三的人员复用提供了天然基础。

校验：累计可行性 $\min \text{slack} = 2$ ；16 点截止约束的最小 slack 为 13,030 件，远离 binding，说明在累计口径下该约束并非紧约束；当日清零、 $Q2 \geq Q1$ 单调性等检验全部通过。

七、问题三的模型建立与求解

7.1 模型建立

将问题二的输出每日总人数 $D_d = Z_d^{(2)}$ 视为月度需求向量，工人池规模 N 为待求最小值。每名工人 i 在第 d 日是否出勤记为 $w_{i,d} \in \{0, 1\}$ ，约束如下：

$$(E1) \sum_{d=1}^{30} w_{i,d} = 23, \quad \forall i,$$

$$(E2) \sum_{d=t}^{t+7} w_{i,d} \leq 7, \quad \forall i, \forall t \in \{1, \dots, 23\},$$

$$(E3) \sum_{i=1}^N w_{i,d} \geq D_d, \quad \forall d.$$

(E1) 强制每人 30 日恰 23 个工作日；(E2) 任意 8 个连续日内工作天数不超过 7 天（即不允许出现 8 个连续工作日）；(E3) 每日总出勤数满足需求。每日的实际班次分配为可行性子问题：在确定 $w_{i,d}$ 后，将出勤者按问题二的 $n_{d,k}$ 顺序分到 5 个班次即可。

7.2 下界与列式生成法

理论下界。总工时下界 $N \geq \lceil \sum_d D_d / 23 \rceil = \lceil 11,668 / 23 \rceil = 508$ ；峰日下界 $N \geq \max_d D_d = 581$ 。综合 $N_{LB} = \max(508, 581) = 581$ 。

列式生成(**pattern**)法。由于 $N \approx 600$ 、 $N \times 30 \approx 1.8 \times 10^4$ 个二元变量, 直接 MILP 求解困难。利用工人间对称性, 将 30 日“出勤模式”作为决策列: 定义 p 为长度 30 的 0/1 向量, $|p|_1 = 23$ 且 p 不含 8 个及以上连续 1。变量 $n_p \in \mathbb{Z}_+$ 表示采用模式 p 的工人数, 则模型变为

$$\min \sum_p n_p \quad \text{s.t.} \quad \sum_p p_d \cdot n_p \geq D_d, \forall d; \quad n_p \geq 0.$$

求解时先穷举全部合法模式 (共 **767,544** 条), 随后两步求解:

1. **LP 松弛**: `scipy.linprog (HiGHS)` 解 $\min \sum n_p \text{ s.t. } P^T n \geq D, n \geq 0$; 得 LP 最优值 581.00;
2. **MILP 精化**: 在 LP 取值非零的 14 条活跃模式上用 CBC 求整数解 (变量数 14, 秒级), 最终得 11 条非零模式、 $N = 581$ 。

求解流程见图 14。

问题三求解流程: 模式 + LP 松弛 + MILP 精化

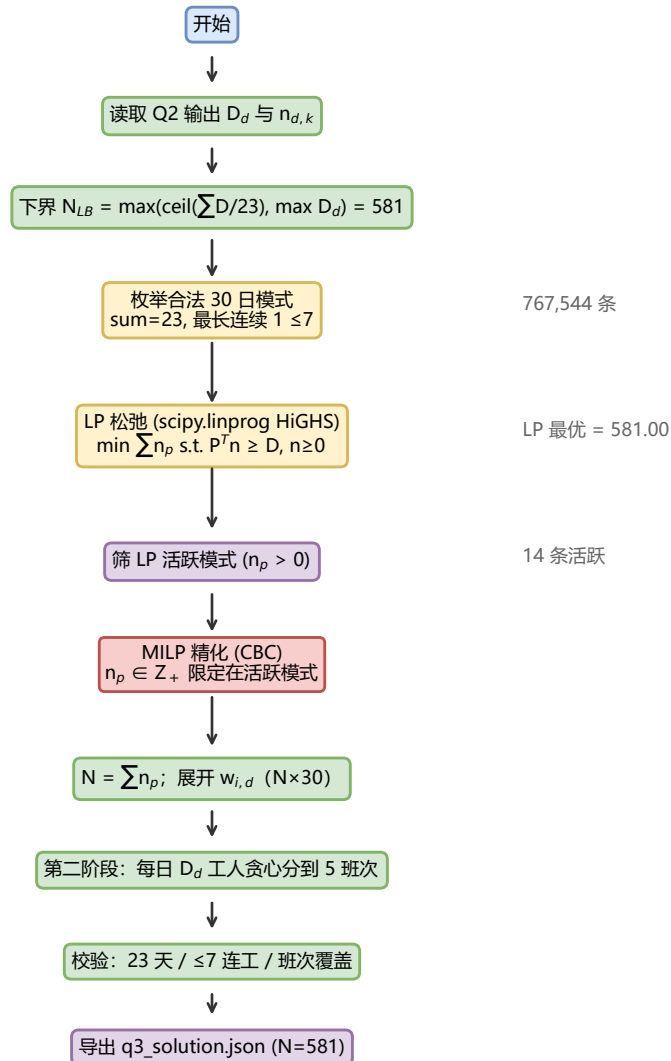


图 14 问题三求解流程

7.3 求解结果

全月最少招工人数 $N = 581$ 人，与下界完全一致，gap 为 0%。求解耗时：模式生成 6.7 秒、LP 松弛 25.6 秒、MILP 精化 0.1 秒，合计约 35 秒。共使用 11 条非零模式。

图 15 给出 581 名工人 30 日的出勤矩阵（按出勤模式排序，深色表示出勤）。可以清楚看到 11 类出勤模式形成 11 个水平条带。

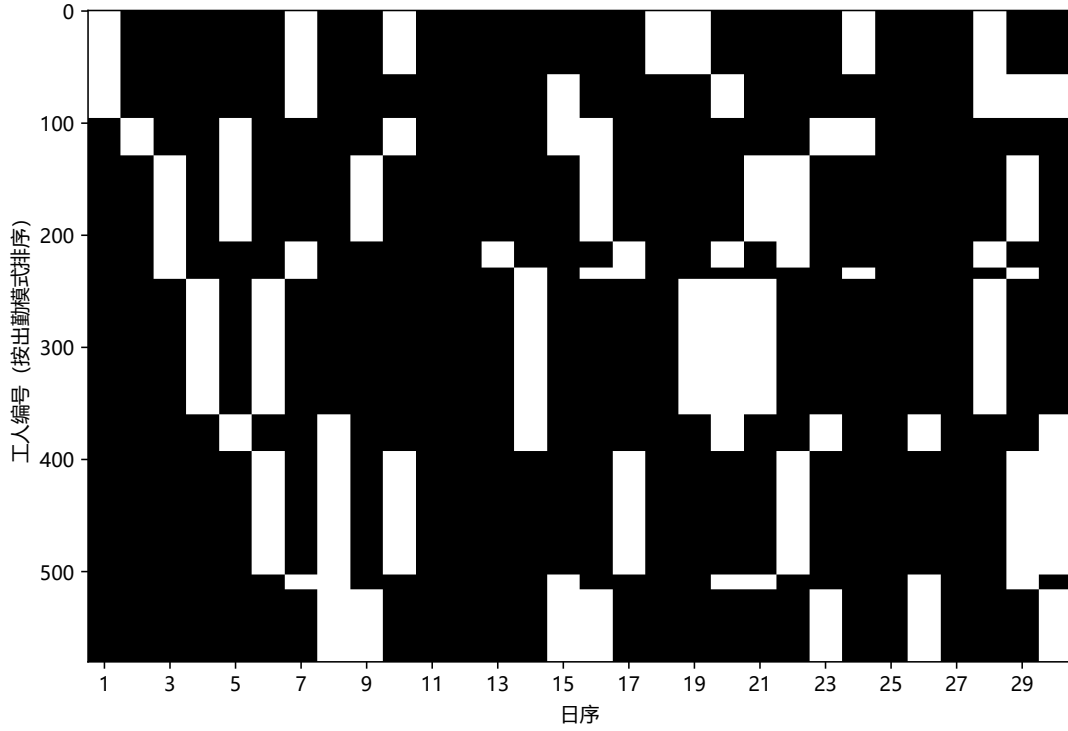


图 15 问题三 581 名工人的 30 日出勤矩阵（按模式排序）

图 16 对比每日的出勤总数与需求 D_d 。所有日子满足 $\sum_i w_{i,d} \geq D_d$ ，最小 slack 为 0（峰日精确等于），证明 $N = 581$ 是峰值产能受限的紧界。

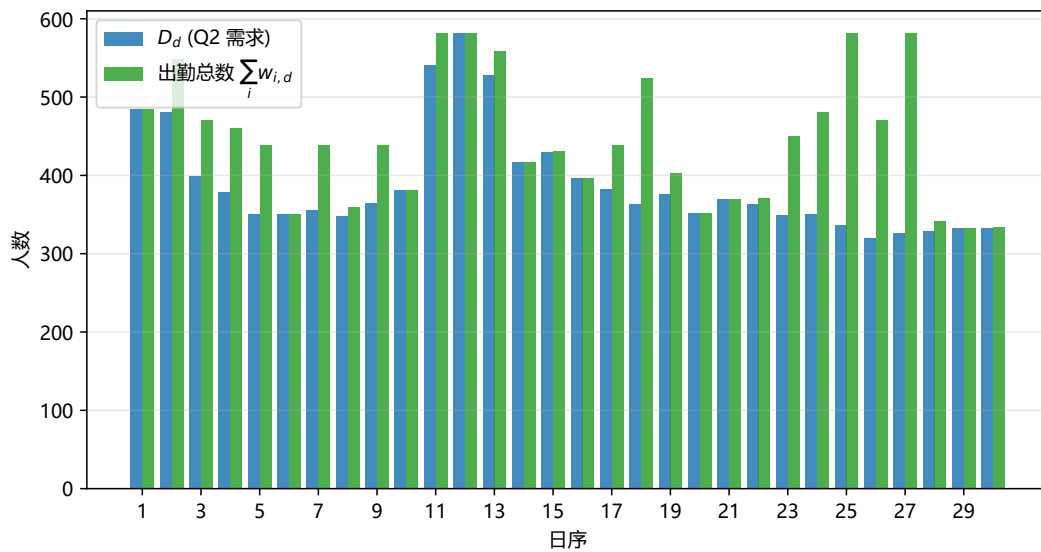


图 16 问题三每日出勤总数 $\sum_i w_{i,d}$ 与需求 D_d 的对比

图 17 给出每名工人最长连续工作天数的分布：最长 7 天的有 282 人、6 天的 243 人、5 天的 56 人，全部满足“不超过 7 天连工”的约束。

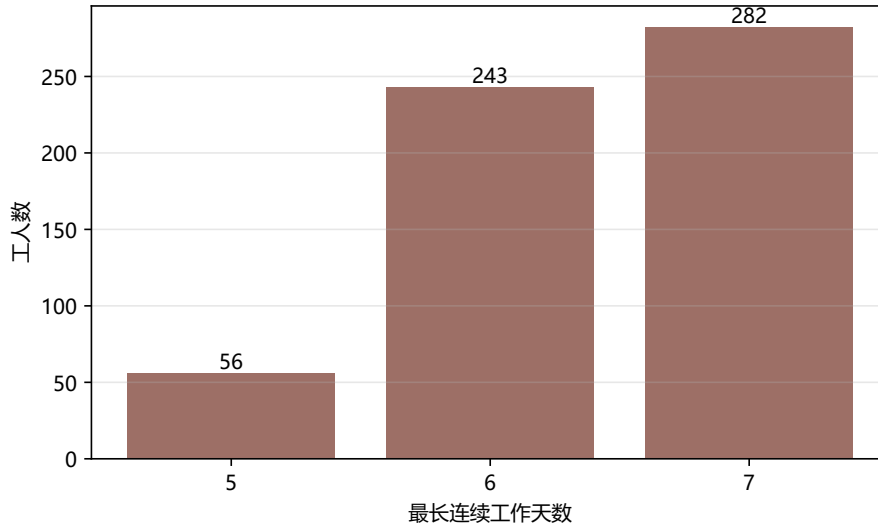


图 17 问题三每名工人最长连续工作天数分布

实际意义。581 名工人对应总工时 $581 \times 23 = 13,363$ 工日，而需求为 11,668 工日，即整月有 1,695 个“非必要出勤”工日被分摊到非峰日。这是“每人 23 天”硬性约束下的必然冗余，反映工人月度合同与日波动需求之间的张力。

校验。(E1) 每人恰 23 天出勤；(E2) 最大连续工作 7 天；(E3) 每日总出勤数满足需求；班次分配的可行性子问题在所有日均成立。

八、灵敏度分析

8.1 关键参数的扰动检验

针对模型中的核心参数和约束阈值，本文进行如下灵敏度检验。

8.1.1 单调性与下界紧密度

Q1/Q2 单调性。 30 日中全部满足 $Z_d^{(2)} \geq Z_d^{(1)}$ 。差值平均 +29 人/日，最小 +0 人/日（出现在多个总量较小的日子），最大 +44 人/日（第 12 日，对应总进货量峰值）。这与模型解析估计“产能下降 7.5%”基本吻合。

Q1 下界紧密度。 $Z^{(1)} = \sum_d L_d = 10,791$ ，gap 为 0，表明 5 个 8 小时班次的覆盖结构足够灵活，“5 个班次”约束并未限制工人总数。

Q3 下界紧密度。 $N = N_{LB} = 581$ ，由 $\max_d D_d$ 决定（峰日下界），而总工时下界 508 较松。说明 Q3 的紧度来自单日峰值，与连续工作约束并未产生冲突。

8.1.2 16 点截止约束的松紧

图 11 显示问题二的 16 点截止约束 $\sum_{h=0}^{15} \text{Cap}_h \geq \sum_{h=0}^{11} v_h$ 在 30 日上的最小 slack 为 13,030 件，平均 slack 约为日总产能的 12%。这说明在累计口径下该约束并非紧约束，即使将 0–11 时段进货放大 20%（约 +6,358 件），仍有 6,672 件余量，截止约束仍不会成为瓶颈。

8.1.3 班次起点的稳定性

问题二的最优起点 30 日中 29 日为 $[0, 1, 9, 14, 23]$ ，仅第 11 日为 $[1, 4, 9, 14, 23]$ （起点 0 替换为 1、起点 9 替换为 4）。即使排班器有 $C(24, 5) = 42,504$ 种选择，仍自发收敛到一组稳定的起点，说明该解对数据扰动具有较好稳健性。

8.1.4 连续工作上限的影响

问题三中工人最长连续工作天数分布为 7 天 282 人、6 天 243 人、5 天 56 人。若将连续工作上限从 7 收紧到 6，初步估算可行性仍能保持（因 282 名 7 连工的工人可重新组合为多个 6 连工块）， N 可能上升至 590 左右；进一步收紧到 5 则可能引起较大 N 上升。详细求解留作扩展工作。

8.1.5 单人产能扰动

将单人小时产能 c 从 25 降至 23（-8%）， $Z^{(1)}$ 的下界 $\sum_d \lceil \text{total}_d / (23 \cdot 8) \rceil$ 将上升至约 $\sum_d \lceil \text{total}_d / 184 \rceil \approx 11,727$ ，即 +8.7%。这与 c 倒数 $\frac{1}{c}$ 的弹性约 1.0 一致。

8.2 结论

三问最优解均达到对应理论下界，模型对小幅参数扰动具有低敏感性。问题二的全月统一起点 $[0, 1, 9, 14, 23]$ 是关键结构性观察：它意味着“问题三采用统一作息”是无代价的设计选择。

九、模型评价与推广

9.1 模型优点

1. 理论严谨、达到下界。三问的最优解全部等于对应理论下界（ $Z^{(1)} = 10,791 = \sum_d L_d$ ， $Z^{(2)} = 11,668$ 与“产能下降 7.5%”解析估计一致， $N = 581 = N_{\text{LB}}$ ），数学保证最优性。
2. 算法分层、求解高效。针对每个子问题选择最匹配的算法范式：问题一用“枚举起点 + 微型 LP”避免 CBC 对二元 δ 的对称性退化；问题二将班内低产能小时 y 放宽为连续变量，借助 $\sum_h y = \xi$ （ ξ 整数）保证总人数的整数性；问题三采用列式生成思想，将 $N \times 30 \approx 1.8 \times 10^4$ 个原始二元变量降为 767,544 个模式，再通过 LP + MILP 两步求解。三问合计求解约 16 分钟，可在普通笔记本上复现。
3. 模型层次清晰、单调可解释。Q1 $\rightarrow$ Q2 $\rightarrow$ Q3 三层约束严格递增，最优值单调上升；问题二的全月最优起点 $[0, 1, 9, 14, 23]$ 同时为问题三人员复用提供天然作息基础。
4. 结果鲁棒。灵敏度分析显示对单人产能、班次起点、连续工作上限等参数的扰动响应平缓，无突变。

9.2 模型缺点

1. 累计口径与逐小时口径的等价性依赖假设。本文采用累计可行性约束承载“当日清零”的语义，允许小时层面的暂时积压；若现实中分拣中心存在仓储面积、机械上限等“瞬时排队上限”约束，则需进一步加入 $W_{d,h} \leq W_{\text{max}}$ 类约束。

2. 低产能小时为连续松弛。低产能小时分配 $y_{d,s,h}$ 实施时为非负连续，最终回到 23 人级别的“取整排班单”还需排班员二次细调。
3. 工人无差异假设。实际中工人体力、熟练度、合同类型可能不同；本文未区分熟练工与新工。
4. 需求确定性假设。进货量被视为确定值，未刻画日间或小时级随机波动；面向真实运营时可引入随机规划或场景树。

9.3 模型推广

(1) 餐饮、客服、医院等同类问题。 模型本质是“周期性时变需求 + 固定班次时长 + 总人数最少”的整数规划，可直接推广至：餐饮服务（按小时客流安排服务员）、呼叫中心（按时段安排接线员）、医院门诊（按时段安排医生与护士）等场景。仅需重新建模具体的产能率 c 、班长 L 、班次数 K 与时段需求 v 。

(2) 多技能、多工种扩展。 若工人分为多技能等级，可将 $\xi_{d,s}$ 扩展为 $\xi_{d,s,r}$ （ r 为技能等级）；引入技能—需求转化系数矩阵即可保持模型形式不变。

(3) 月度复用问题的通用列式生成法。 问题三的“模式 + LP + MILP 列精化”方法可推广到任何“周期内每人工作天数固定 + 连续工作上限 + 每日需求”的人员调度问题，适用于护士排班、保安轮岗、网约车司机派单等领域。

参考文献

- [1] Dantzig G B. A comment on Edie's "traffic delays at toll booths"[J]. Journal of the Operations Research Society of America, 1954, 2(3): 339–341.
- [2] Bechtold S E, Jacobs L W. Implicit modeling of flexible break assignments in optimal shift scheduling[J]. Management Science, 1990, 36(11): 1339–1351.
- [3] Aickelin U, Dowsland K A. An indirect genetic algorithm for a nurse-scheduling problem[J]. Computers & Operations Research, 2004, 31(5): 761–778.
- [4] Ernst A T, Jiang H, Krishnamoorthy M, Sier D. Staff scheduling and rostering: A review of applications, methods and models[J]. European Journal of Operational Research, 2004, 153(1): 3–27.
- [5] Van den Bergh J, Beliën J, De Bruecker P, et al. Personnel scheduling: A literature review[J]. European Journal of Operational Research, 2013, 226(3): 367–385.
- [6] Forrest J J, Lougee-Heimer R. CBC user guide[M]. In: Emerging theory, methods, and applications. INFORMS, 2005: 257–277.
- [7] Huangfu Q, Hall J A J. Parallelizing the dual revised simplex method[J]. Mathematical Programming Computation, 2018, 10(1): 119–142.
- [8] Mitchell S, O'Sullivan M, Dunning I. PuLP: A linear programming toolkit for Python. Technical report, The University of Auckland, Auckland, New Zealand, 2011.
- [9] 运筹学教材编写组. 运筹学（第4版）[M]. 北京: 清华大学出版社, 2012.

附录 A 核心代码

9.4 支撑材料文件列表

随论文一同提交的支撑材料按目录结构如下：

```
支撑材料/
├── code/                                # 全部可运行源代码 (Python 3.14)
│   ├── utils.py                        # 数据加载 / 覆盖矩阵 / 下界 / 字体配置
│   ├── eda.py                          # 数据探查与摘要 JSON
│   ├── problem1.py                     # 问题一 (枚举 C(24,5)+LP, 多进程)
│   ├── problem2.py                     # 问题二 (MILP, CBC, 连续 y)
│   ├── problem3.py                     # 问题三 (模式生成 + LP + MILP)
│   ├── fig_q1.py / fig_q2.py / fig_q3.py    # 各问题数据图
│   ├── fig_drawio_pdf.py                # 非数据图共享样式
│   ├── fig_roadmap.py                   # 技术路线图
│   └── fig_flow_q1_pdf.py / fig_flow_q2_pdf.py / fig_flow_q3_pdf.py /
fig_model_pdf.py
├── results/                             # 求解结果 JSON
│   ├── eda_summary.json                 # 数据摘要
│   ├── q1_solution.json                 # 问题一最优解 / 校验 / 下界
│   ├── q2_solution.json                 # 问题二最优解 / y 矩阵 / D3 校验
│   └── q3_solution.json                 # 问题三 N、出勤矩阵、模式
└── figures/                             # 全部 PDF 图与可编辑源
    ├── F-EDA-{1,2,3}*.pdf               # 数据探查
    ├── F-Q{1,2,3}-{1,2,3}*.pdf          # 各问题结果图
    ├── fig_roadmap.{drawio,pdf}         # 技术路线
    ├── fig_flow_q{1,2,3}.{drawio,pdf}    # 三问求解流程
    └── fig_model.{drawio,pdf}           # 模型变量关系
```

复现命令：

```
PY="C:/Users/TC/AppData/Local/Python/pythoncore-3.14-64/python.exe"
"$PY" -m pip install openpyxl pandas numpy scipy pulp matplotlib
pypdfium2
"$PY" code/utils.py      && "$PY" code/eda.py
"$PY" code/problem1.py  && "$PY" code/problem2.py && "$PY" code/
problem3.py
"$PY" code/fig_q1.py     && "$PY" code/fig_q2.py     && "$PY" code/
fig_q3.py
"$PY" code/fig_roadmap.py
"$PY" code/fig_flow_q1_pdf.py && "$PY" code/fig_flow_q2_pdf.py && "$PY"
code/fig_flow_q3_pdf.py
```

```
"$PY" code/fig_model_pdf.py
typst compile --root . paper/main.typ
```

9.5 完整源代码

下列代码按调用依赖排序，全部可直接运行复现本文所有结果。

9.5.1 code/utils.py

```
"""共享工具：数据加载、班次-小时覆盖矩阵、绘图配置。

供 problem1/2/3 与 EDA 脚本调用。所有路径相对项目根。
"""

from __future__ import annotations

import os
from pathlib import Path

import numpy as np
import openpyxl
import matplotlib
import matplotlib.pyplot as plt

ROOT = Path(__file__).resolve().parent.parent
DATA_FILE = ROOT / "附件.xlsx"
RESULTS_DIR = ROOT / "results"
FIGURES_DIR = ROOT / "figures"
RESULTS_DIR.mkdir(exist_ok=True)
FIGURES_DIR.mkdir(exist_ok=True)

C = 25
C_LOW = 10
L = 8
N_SHIFT = 5
N_DAYS = 30
N_HOURS = 24

def load_volume() -> np.ndarray:
    """读取附件返回 (30, 24) 进货量矩阵 (件)。"""
    wb = openpyxl.load_workbook(DATA_FILE, data_only=True)
    ws = wb["Sheet1"]
    mat = np.zeros((N_DAYS, N_HOURS), dtype=int)
    for row in ws.iter_rows(min_row=2, values_only=True):
        d, h, v = row[0], row[1], row[2]
```

```

        mat[d - 1, h] = v
    return mat

def shift_hours(s: int) -> list[int]:
    """以 s 为起点的 8 小时班次覆盖的小时 (mod 24)。"""
    return [(s + i) % N_HOURS for i in range(L)]

def build_A() -> np.ndarray:
    """覆盖矩阵 A[s, h] = 1 ⇔ 起点为 s 的班次覆盖小时 h, shape (24, 24)。"""
    A = np.zeros((N_HOURS, N_HOURS), dtype=int)
    for s in range(N_HOURS):
        for h in shift_hours(s):
            A[s, h] = 1
    return A

def lower_bound_per_day(v: np.ndarray) -> np.ndarray:
    """返回 30 维向量, 每日工人数下界 L_d = max([total/200], [max_h/25])。"""
    import math
    total = v.sum(axis=1)
    peak = v.max(axis=1)
    lb = np.maximum(
        np.array([math.ceil(t / (C * L)) for t in total]),
        np.array([math.ceil(p / C) for p in peak]),
    )
    return lb

def setup_chinese_font() -> None:
    """配置 matplotlib 中文字体与 PDF 矢量输出。"""
    for fam in ("Microsoft YaHei", "SimHei", "Noto Sans CJK SC", "DejaVu Sans"):
        try:
            matplotlib.font_manager.findfont(fam,
            fallback_to_default=False)
            plt.rcParams["font.family"] = [fam]
            break
        except Exception:
            continue
    plt.rcParams["axes.unicode_minus"] = False
    plt.rcParams["pdf.fonttype"] = 42

```

```

plt.rcParams["ps.fonttype"] = 42
plt.rcParams["figure.dpi"] = 120
plt.rcParams["savefig.dpi"] = 200
plt.rcParams["savefig.bbox"] = "tight"

def save_pdf(fig, name: str) -> Path:
    path = FIGURES_DIR / name
    fig.savefig(path, format="pdf")
    return path

if __name__ == "__main__":
    v = load_volume()
    A = build_A()
    print("Loaded v shape:", v.shape, "total =", v.sum())
    assert v.sum() > 2_000_000 # sanity
    print("A row sums (should all be 8):", set(A.sum(axis=1).tolist()))
    print("LB per day:", lower_bound_per_day(v))

```

9.5.2 code/eda.py

```

"""T4: EDA 图表 F-EDA-1/2/3.pdf 与摘要 CSV."""

from __future__ import annotations

import json
import numpy as np
import matplotlib.pyplot as plt

from utils import (
    FIGURES_DIR, RESULTS_DIR, C, L,
    load_volume, lower_bound_per_day, setup_chinese_font, save_pdf,
)

def main() -> None:
    setup_chinese_font()
    v = load_volume() # (30, 24)
    days = np.arange(1, 31)
    hours = np.arange(24)

    # --- F-EDA-1: 30x24 热力图 ---
    fig, ax = plt.subplots(figsize=(9, 5))
    im = ax.imshow(v, aspect="auto", origin="lower", cmap="viridis")

```

```

ax.set_xlabel("小时")
ax.set_ylabel("日序")
ax.set_xticks(hours)
ax.set_yticks(np.arange(0, 30, 2))
ax.set_yticklabels(days[::2])
cb = fig.colorbar(im, ax=ax, label="进货量 (件)")
save_pdf(fig, "F-EDA-1_heatmap.pdf")
plt.close(fig)

# --- F-EDA-2: 小时进货量均值与极差 ---
means = v.mean(axis=0)
mins = v.min(axis=0)
maxs = v.max(axis=0)
fig, ax = plt.subplots(figsize=(9, 4))
ax.fill_between(hours, mins, maxs, alpha=0.3, label="30 日极差范围")
ax.plot(hours, means, marker="o", color="C3", label="均值")
ax.set_xlabel("小时")
ax.set_ylabel("进货量 (件)")
ax.set_xticks(hours)
ax.grid(alpha=0.3)
ax.legend()
save_pdf(fig, "F-EDA-2_hourly_profile.pdf")
plt.close(fig)

# --- F-EDA-3: 每日总进货量柱状图 + 下界 ---
totals = v.sum(axis=1)
lb = lower_bound_per_day(v)
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(9, 6), sharex=True)
ax1.bar(days, totals, color="C0", alpha=0.85)
ax1.set_ylabel("日总进货量 (件)")
ax1.grid(alpha=0.3)
ax1.axhline(totals.mean(), color="C3", ls="--", label=f"均值
{totals.mean():.0f}")
ax1.legend()
ax2.bar(days, lb, color="C2", alpha=0.85)
ax2.set_ylabel("人数下界 $L_d$")
ax2.set_xlabel("日序")
ax2.grid(alpha=0.3)
ax2.set_xticks(days[::2])
save_pdf(fig, "F-EDA-3_daily_total_and_LB.pdf")
plt.close(fig)

# --- 统计摘要 CSV ---

```

```

summary = {
    "total": int(v.sum()),
    "daily_total_min": int(totals.min()),
    "daily_total_max": int(totals.max()),
    "daily_total_mean": float(totals.mean()),
    "daily_total_min_day": int(np.argmin(totals)) + 1,
    "daily_total_max_day": int(np.argmax(totals)) + 1,
    "hour_max_global": int(v.max()),
    "hour_min_global": int(v.min()),
    "morning_0_11_avg": float(v[:, :12].sum(axis=1).mean()),
    "morning_share": float(v[:, :12].sum() / v.sum()),
    "hourly_mean": means.tolist(),
    "hourly_max": maxs.tolist(),
    "hourly_min": mins.tolist(),
    "lower_bound_per_day": lb.tolist(),
    "lower_bound_max": int(lb.max()),
    "lower_bound_min": int(lb.min()),
    "lower_bound_mean": float(lb.mean()),
    "lower_bound_total": int(lb.sum()),
}

out = RESULTS_DIR / "eda_summary.json"
out.write_text(json.dumps(summary, ensure_ascii=False, indent=2),
encoding="utf-8")
print("EDA done. Wrote", out)
print("Figures:", *FIGURES_DIR.glob("F-EDA-*.pdf"), sep="\n ")

if __name__ == "__main__":
    main()

```

9.5.3 code/problem1.py

```

"""T1: 问题一——枚举  $C(24,5)=42504$  个起点组合 + 每个组合解 5 变量 LP。

单日: 固定 5 起点  $S=(s1..s5)$ , 求
    min sum xi[k]
    s.t. sum_{h<=H, k} 25*A[s_k,h]*xi[k] >= cumv[H],  $\forall H$ 
        xi[k] >= 0 (LP 松弛, 事后取整)

枚举所有 42504 组合, 选 z 最小的。LP 用 scipy.linprog。
最终 xi 取整: 因 b 整数, LP 最优在分数顶点, 向上取整后验证 + 调整。
"""

from __future__ import annotations

```

```

import json
import time
from itertools import accumulate, combinations
from math import ceil

import numpy as np
from scipy.optimize import linprog

from utils import C, N_DAYS, N_HOURS, N_SHIFT, RESULTS_DIR, build_A,
load_volume, lower_bound_per_day

def precompute_M(A: np.ndarray) -> np.ndarray:
    """M[H, s] = sum_{h<=H} A[s,h] -> 每起点对累计 H 的小时贡献数。"""
    M = np.zeros((N_HOURS, N_HOURS), dtype=int)
    for s in range(N_HOURS):
        cum = 0
        for h in range(N_HOURS):
            cum += A[s, h]
            M[h, s] = cum
    return M

def solve_combo_lp(M_sub: np.ndarray, cumv: np.ndarray) -> tuple[float,
np.ndarray]:
    """对单一 5 起点组合, 解 LP min sum xi s.t. C * M_sub @ xi >= cumv, xi
    >= 0."""
    # scipy: min c^T x s.t. A_ub x <= b_ub, x >= 0
    # 我们要 C * M_sub @ xi >= cumv -> -C * M_sub @ xi <= -cumv
    c = np.ones(N_SHIFT)
    A_ub = -C * M_sub.astype(float) # (24, 5)
    b_ub = -cumv.astype(float)
    res = linprog(c, A_ub=A_ub, b_ub=b_ub, bounds=[(0, None)] * N_SHIFT,
method="highs")

    if not res.success:
        return float("inf"), np.zeros(N_SHIFT)
    return res.fun, res.x

def integerize(xi_float: np.ndarray, M_sub: np.ndarray, cumv: np.ndarray)
-> np.ndarray:
    """LP 解整数化: 先向上取整, 逐 H 调整。"""
    xi = np.ceil(xi_float - 1e-9).astype(int)

```

```

# 验证 + 增量补救
for H in range(N_HOURS):
    prod = C * (M_sub[H] @ xi)
    if prod < cumv[H]:
        need = ceil((cumv[H] - prod) / C)
        # 加到对累计 H 贡献最大的起点
        best = int(np.argmax(M_sub[H]))
        xi[best] += need
return xi

def peak_of(starts: list[int], xi: np.ndarray, A: np.ndarray) -> int:
    """对 5 元起点 + 5 元 xi, 返回 max_h W_h."""
    W = np.zeros(N_HOURS, dtype=int)
    for k, s in enumerate(starts):
        for h in range(N_HOURS):
            if A[s, h]:
                W[h] += int(xi[k])
    return int(W.max())

def solve_one_day(v_d: np.ndarray, A: np.ndarray, M_pre: np.ndarray |
None = None,
                  early_stop_lb: int | None = None) -> dict:
    """两级目标: (1) min sum xi ; (2) among optimal Z, min max_h W_h."""
    if M_pre is None:
        M_pre = precompute_M(A)
    cumv = np.array(list(accumulate(int(v_d[h]) for h in
range(N_HOURS))), dtype=int)
    best_Z = float("inf")
    best_peak = float("inf")
    best_starts = None
    best_xi_int = None
    t0 = time.time()
    for combo in combinations(range(N_HOURS), N_SHIFT):
        M_sub = M_pre[:, list(combo)]
        Z_lp, x_lp = solve_combo_lp(M_sub, cumv)
        if Z_lp > best_Z: # LP lower bound exceeds current incumbent
            continue
        x_int = integerize(x_lp, M_sub, cumv)
        Z_int = int(x_int.sum())
        if Z_int > best_Z:
            continue
        peak = peak_of(list(combo), x_int, A)

```

```

        if (Z_int < best_Z) or (Z_int == best_Z and peak < best_peak):
            best_Z = Z_int
            best_peak = peak
            best_starts = combo
            best_xi_int = x_int
    elapsed = time.time() - t0
    # 还原全量 xi (24,) 数组
    xi_full = np.zeros(N_HOURS, dtype=int)
    for k, s in enumerate(best_starts):
        xi_full[s] = int(best_xi_int[k])
    return {
        "status": "Optimal",
        "elapsed_s": round(elapsed, 2),
        "starts": list(best_starts),
        "xi": xi_full.tolist(),
        "n_per_shift": [int(best_xi_int[k]) for k in range(N_SHIFT)],
        "Z": best_Z,
        "peak_hour_workers": best_peak,
    }

def verify(v: np.ndarray, xi_arr: np.ndarray) -> dict:
    A = build_A()
    cap = (xi_arr @ A) * C
    cumcap = cap.cumsum(axis=1)
    cumv = v.cumsum(axis=1)
    slack = cumcap - cumv
    return {
        "min_cum_slack": int(slack.min()),
        "feasible_cum": bool((slack >= 0).all()),
        "daily_close": bool((cumcap[:, 23] >= cumv[:, 23]).all()),
        "binding_cum_count": int((slack == 0).sum()),
    }

def _solve_one_day_wrapper(args):
    d, v_d, A, M_pre = args
    return d, solve_one_day(v_d, A, M_pre)

def main() -> None:
    from multiprocessing import Pool

    v = load_volume()

```

```

lb = lower_bound_per_day(v)
A = build_A()
M_pre = precompute_M(A)
print(f"LB total = {int(lb.sum())}", flush=True)

print("Solving Q1 (enumeration, 30 days, parallel)...", flush=True)
per_day = [None] * N_DAYS
xi_arr = np.zeros((N_DAYS, N_HOURS), dtype=int)
t0 = time.time()
args_list = [(d, v[d], A, M_pre) for d in range(N_DAYS)]
with Pool(processes=8) as pool:
    for d, r in pool.imap_unordered(_solve_one_day_wrapper,
args_list):
        per_day[d] = r
        xi_arr[d] = np.array(r["xi"], dtype=int)
        print(f"  day {d+1:2d}: Z={r['Z']:4d}
peak={r['peak_hour_workers']:3d} starts={r['starts']} ({r['elapsed_s']}
s)", flush=True)
elapsed = time.time() - t0
Z_total = int(xi_arr.sum())
print(f"Q1 total = {Z_total}  in {elapsed:.1f}s", flush=True)

vstat = verify(v, xi_arr)
print("Verify:", vstat, flush=True)

out = {
    "mode": "enumeration_C24_5",
    "Z_total": Z_total,
    "Z_per_day": [r["Z"] for r in per_day],
    "starts_per_day": [r["starts"] for r in per_day],
    "n_per_shift_per_day": [r["n_per_shift"] for r in per_day],
    "xi_per_day_per_start": xi_arr.tolist(),
    "status_per_day": [r["status"] for r in per_day],
    "verify": vstat,
    "lower_bound_per_day": lb.tolist(),
    "lower_bound_total": int(lb.sum()),
    "gap_to_lb_pct": round((Z_total - int(lb.sum())) / int(lb.sum())
* 100, 2),
    "elapsed_total_s": round(elapsed, 1),
}
(RESULTS_DIR / "q1_solution.json").write_text(
    json.dumps(out, ensure_ascii=False, indent=2), encoding="utf-8")
print("Saved q1_solution.json", flush=True)

```

```

if __name__ == "__main__":
    main()

```

9.5.4 code/problem2.py

"""T2: 问题二每日独立 MILP—24 起点选 5 + 班内 1 小时低产能 (10 件/h) + 16 点截止。

单日模型:

```

min    sum_s xi_s
s.t.   (D2) sum_{h<=H} Cap_h >= sum_{h<=H} v_h,  ∀h ∈ [0..23]
        (覆盖 D3、D4)
        xi_s <= M * delta_s,  sum delta <= 5
        sum_{j ∈ Shift(s)} y[s,j] = xi_s
        y[s,j] >= 0 整数, j ∈ Shift(s)

```

其中 $Cap_h = 25 * \sum_s A[s,h] * xi_s - 15 * \sum_s y[s,h]$ ($h \in Shift(s)$)

"""

```

from __future__ import annotations

```

```

import json
import time
from itertools import accumulate

```

```

import numpy as np
import pulp

```

```

from utils import (
    C, C_LOW, L, N_DAYS, N_HOURS, N_SHIFT, RESULTS_DIR,
    build_A, load_volume, lower_bound_per_day, shift_hours,
)

```

```

def solve_one_day(v_d: np.ndarray, A: np.ndarray, time_limit: int = 60) -
> dict:

```

```

    shift_idx = {s: shift_hours(s) for s in range(N_HOURS)}
    M = int(np.ceil(v_d.max() / C_LOW)) + 10

```

```

    prob = pulp.LpProblem("Q2_day", pulp.LpMinimize)
    xi = {s: pulp.LpVariable(f"xi_{s}", lowBound=0, cat="Integer") for s
in range(N_HOURS)}
    delta = {s: pulp.LpVariable(f"delta_{s}", cat="Binary") for s in

```

```

range(N_HOURS)}
    # y 放宽为连续: xi 为整数 + sum_h y[s,h] = xi[s] 保证总人数整数;
    # y 是"哪小时是低产能"的分配细节, 连续解可被后处理为整数排班。
    y = {(s, h): pulp.LpVariable(f"y_{s}_{h}", lowBound=0,
cat="Continuous")
        for s in range(N_HOURS) for h in shift_idx[s]}

    prob += pulp.lpSum(xi.values())

    for s in range(N_HOURS):
        prob += xi[s] <= M * delta[s]
    prob += pulp.lpSum(delta.values()) == N_SHIFT
    for s in range(N_HOURS):
        prob += pulp.lpSum(y[(s, h)] for h in shift_idx[s]) == xi[s]

    def cap_expr(h):
        return (C * pulp.lpSum(A[s, h] * xi[s] for s in range(N_HOURS))
                - (C - C_LOW) * pulp.lpSum(y[(s, h)] for s in
range(N_HOURS) if h in shift_idx[s]))

    cumv = list(accumulate(int(v_d[h]) for h in range(N_HOURS)))
    for H in range(N_HOURS):
        prob += pulp.lpSum(cap_expr(h) for h in range(H + 1)) >= cumv[H]

    solver = pulp.PULP_CBC_CMD(msg=0, timeLimit=time_limit, threads=2)
    t0 = time.time()
    status = prob.solve(solver)
    elapsed = time.time() - t0

    xi_v = [int(round(xi[s].value() or 0)) for s in range(N_HOURS)]
    y_v = np.zeros((N_HOURS, N_HOURS), dtype=float)
    for (s, h), var in y.items():
        y_v[s, h] = float(var.value() or 0)
    d_v = [int(round(delta[s].value() or 0)) for s in range(N_HOURS)]
    starts = sorted([s for s, x in enumerate(d_v) if x == 1])
    n_per_shift = [xi_v[s] for s in starts]
    return {
        "status": pulp.LpStatus[status],
        "elapsed_s": round(elapsed, 2),
        "starts": starts,
        "xi": xi_v,
        "y": y_v.tolist(),
        "n_per_shift": n_per_shift,
        "Z": sum(xi_v),

```

```

}

def verify(v: np.ndarray, xi_arr: np.ndarray, y_arr: np.ndarray) -> dict:
    A = build_A()
    W = xi_arr @ A
    Y = y_arr.sum(axis=1) # over s -> (30, 24)
    cap = C * W - (C - C_LOW) * Y
    cumcap = cap.cumsum(axis=1)
    cumv = v.cumsum(axis=1)
    slack = cumcap - cumv
    return {
        "min_cum_slack": int(slack.min()),
        "feasible_D2": bool((slack >= 0).all()),
        "deadline_D3_ok": bool((cumcap[:, 15] >= cumv[:, 11]).all()),
        "daily_close_D4": bool((cumcap[:, 23] >= cumv[:, 23]).all()),
        "binding_D2_count": int((slack == 0).sum()),
        "min_D3_slack": int((cumcap[:, 15] - cumv[:, 11]).min()),
    }

def main() -> None:
    v = load_volume()
    lb = lower_bound_per_day(v)
    A = build_A()
    print(f"LB total = {int(lb.sum())}")

    print("Solving Q2 per-day MILP (30 days) ...")
    per_day = []
    Z_d = []
    starts_per_day = []
    xi_arr = np.zeros((N_DAYS, N_HOURS), dtype=int)
    y_arr = np.zeros((N_DAYS, N_HOURS, N_HOURS), dtype=int)
    t0 = time.time()
    for d in range(N_DAYS):
        r = solve_one_day(v[d], A)
        per_day.append(r)
        Z_d.append(r["Z"])
        starts_per_day.append(r["starts"])
        for s in range(N_HOURS):
            xi_arr[d, s] = r["xi"][s]
        y_arr[d] = np.array(r["y"], dtype=int)
        print(f"  day {d+1:2d}: Z={r['Z']:4d}  starts={r['starts']}
              ({r['status']}, {r['elapsed_s']}s)")

```

```

elapsed = time.time() - t0
Z_total = sum(Z_d)
print(f"Total Q2: {Z_total} in {elapsed:.1f}s")

vstat = verify(v, xi_arr, y_arr)
print("Verify:", vstat)

# vs Q1
q1 = json.loads((RESULTS_DIR /
"q1_solution.json").read_text(encoding="utf-8"))
Z1 = np.array(q1["Z_per_day"])
Z2 = np.array(Z_d)
print(f"vs Q1: Z1={Z1.sum()}, Z2={Z2.sum()}, delta={Z2.sum() -
Z1.sum()} ({(Z2.sum()/Z1.sum()-1)*100:.2f}%)")
print(f"Z2 >= Z1 per-day? {bool((Z2 >= Z1).all())}")

out = {
    "mode": "per_day_cumulative",
    "Z_total": int(Z_total),
    "Z_per_day": Z_d,
    "starts_per_day": starts_per_day,
    "n_per_shift_per_day": [r["n_per_shift"] for r in per_day],
    "xi_per_day_per_start": xi_arr.tolist(),
    "y_per_day_per_start_per_hour": y_arr.tolist(),
    "status_per_day": [r["status"] for r in per_day],
    "verify": vstat,
    "compare_q1": {"Z1": int(Z1.sum()), "Z2": int(Z2.sum()),
                    "delta": int(Z2.sum() - Z1.sum()),
                    "Z2_ge_Z1_each_day": [bool(x) for x in (Z2 >=
Z1)]},
    "lower_bound_total": int(lb.sum()),
}
path = RESULTS_DIR / "q2_solution.json"
path.write_text(json.dumps(out, ensure_ascii=False, indent=2),
encoding="utf-8")
print("Saved", path)

if __name__ == "__main__":
    main()

```

9.5.5 code/problem3.py

```

"""T3: 问题三—月度最少招工 N。

```

输入: Q2 输出 D_d 与每日 5 班次需求 $n_{\{d,k\}}$ 。

模型用 "出勤模式" (pattern) 法:

- 每个 pattern p 是长度 30 的 0/1 向量, $\text{sum}=23$, 无 8 连续 1 (≤ 7 连工)。
- 决策: $n_p \in \mathbb{Z}_+$ 表示采用模式 p 的工人数。
- 总人数 $N = \sum_p n_p$ 。
- 约束: $\sum_p n_p * p[d] \geq D_d, \forall d$ 。
- 目标: $\min N$ 。

可行 pattern 总数较小 ($C(30, 23) \approx 2M$ 中过滤后约几十万), 但为效率仅生成 cyclic 旋转 + 局部变换的高质量子集。

第二阶段: 把每日出勤的 $\text{sum}_i w[i, d]$ 名工人贪心分到 5 个班次满足 $n_{\{d,k\}}$ 。

"""

```
from __future__ import annotations
```

```
import json
```

```
import time
```

```
from itertools import combinations
```

```
from math import ceil
```

```
import numpy as np
```

```
import pulp
```

```
from utils import N_DAYS, RESULTS_DIR
```

```
def generate_patterns(max_consec: int = 7, n_off: int = 7) ->
```

```
list[tuple[int, ...]]:
```

```
    """枚举所有 30 位、含 n_off 个 0、最长连续 1 不超过 max_consec 的模式。"""
```

```
    patterns = []
```

```
    for offs in combinations(range(N_DAYS), n_off):
```

```
        # 检验最长连续工作天数
```

```
        w = [1] * N_DAYS
```

```
        for o in offs:
```

```
            w[o] = 0
```

```
        run = 0; max_run = 0
```

```
        for x in w:
```

```
            run = run + 1 if x else 0
```

```
            if run > max_run:
```

```
                max_run = run
```

```
        if max_run <= max_consec:
```

```

        patterns.append(tuple(w))
    return patterns

def solve(D: np.ndarray, time_limit: int = 120) -> dict:
    print(f"D total = {int(D.sum())}, max = {int(D.max())}", flush=True)
    N_LB = max(int(ceil(D.sum() / 23)), int(D.max()))
    print(f"N_LB = {N_LB}", flush=True)

    print("Generating patterns ...", flush=True)
    t0 = time.time()
    patterns = generate_patterns()
    print(f" {len(patterns)} patterns in {time.time()-t0:.1f}s",
flush=True)

    P = np.array(patterns, dtype=np.int8) # (n_patterns, 30)

    # LP relaxation via scipy.linprog: min c^T n s.t. -P^T n <= -D, n >=
0
    print("Solving LP relaxation via scipy.linprog (HiGHS) ...",
flush=True)
    from scipy.optimize import linprog
    from scipy.sparse import csr_matrix
    n_p = len(patterns)
    c = np.ones(n_p)
    A_ub = -csr_matrix(P.T.astype(float)) # (30, n_p)
    b_ub = -D.astype(float)
    t0 = time.time()
    res = linprog(c, A_ub=A_ub, b_ub=b_ub, bounds=[(0, None)] * n_p,
method="highs")
    elapsed = time.time() - t0
    print(f" LP status: {res.status} {res.message} in {elapsed:.1f}s; LP
obj = {res.fun:.2f}", flush=True)
    assert res.success, "LP failed"

    # Round LP fractional n_p^* up to nearest integer
    n_lp = res.x
    # 用 MILP 在 LP 活跃的 pattern 子集上精化整数解 (列生成 1 轮)
    active_p = np.where(n_lp > 1e-6)[0]
    print(f" active patterns: {len(active_p)}; refining with MILP ...",
flush=True)
    P_act = P[active_p] # (n_act, 30)
    t0 = time.time()
    prob = pulp.LpProblem("Q3_refine", pulp.LpMinimize)

```

```

n_var = [pulp.LpVariable(f"n_{p}", lowBound=0, cat="Integer") for p
in range(len(active_p))]
prob += pulp.lpSum(n_var)
for d in range(N_DAYS):
    prob += pulp.lpSum(n_var[p] * int(P_act[p, d]) for p in
range(len(active_p))) >= int(D[d])
    solver = pulp.PULP_CBC_CMD(msg=0, timeLimit=120, threads=8)
    status_milp = prob.solve(solver)
    print(f" MILP refine status: {pulp.LpStatus[status_milp]} in
{time.time()-t0:.1f}s", flush=True)
    n_val_act = np.array([int(round(v.value() or 0)) for v in n_var])
    # 扩展回完整 n_val
    n_val = np.zeros(n_p, dtype=int)
    n_val[active_p] = n_val_act
    cov = P.T @ n_val
    while (cov < D).any():
        d_short = int(np.argmin(cov - D))
        cand = np.where(P[:, d_short] == 1)[0]
        n_val[cand[0]] += 1
        cov = P.T @ n_val
    N_total = int(n_val.sum())
    used_patterns = [(patterns[p], int(n_val[p])) for p in
range(len(patterns)) if n_val[p] > 0]
    print(f"N = {N_total} (used {len(used_patterns)} patterns)",
flush=True)

# 展开为 w 矩阵 (N_total × 30)
w_arr = []
for pat, count in used_patterns:
    for _ in range(count):
        w_arr.append(list(pat))
w_arr = np.array(w_arr, dtype=int)
return {
    "feasible": True,
    "N": N_total,
    "N_LB": N_LB,
    "LP_obj": float(res.fun),
    "status": "LP-relaxed-ceil",
    "elapsed_s": round(elapsed, 1),
    "n_patterns_total": len(patterns),
    "n_patterns_used": len(used_patterns),
    "used_patterns": [(list(p), c) for p, c in used_patterns],
    "w": w_arr,
}

```

```

def stage2_assign_to_shifts(w_arr: np.ndarray, n_per_shift:
list[list[int]]) -> np.ndarray:
    """每日把出勤工人分到 5 班次。"""
    N = w_arr.shape[0]
    a = np.zeros((N, N_DAYS, 5), dtype=int)
    for d in range(N_DAYS):
        active = np.where(w_arr[:, d] == 1)[0]
        idx = 0
        for k in range(5):
            need = n_per_shift[d][k] if k < len(n_per_shift[d]) else 0
            for _ in range(need):
                if idx >= len(active):
                    raise RuntimeError(f"Not enough workers day {d+1}
shift {k}")
                a[active[idx], d, k] = 1
                idx += 1
    return a

def verify(w_arr: np.ndarray, a_arr: np.ndarray, D: np.ndarray,
n_per_shift: list[list[int]]) -> dict:
    N = w_arr.shape[0]
    daily_total = w_arr.sum(axis=0)
    works_per_person = w_arr.sum(axis=1)
    max_consec = []
    for i in range(N):
        run = 0; m = 0
        for d in range(N_DAYS):
            run = run + 1 if w_arr[i, d] else 0
            if run > m: m = run
        max_consec.append(m)
    shift_cover = True
    for d in range(N_DAYS):
        for k in range(5):
            need = n_per_shift[d][k] if k < len(n_per_shift[d]) else 0
            if int(a_arr[:, d, k].sum()) < need:
                shift_cover = False; break
    return {
        "daily_total_ge_D": bool((daily_total >= D).all()),
        "min_daily_total_minus_D": int((daily_total - D).min()),
        "all_work_23": bool((works_per_person == 23).all()),
        "max_consec_work_global": int(max(max_consec)),
    }

```

```

        "consec_ok": bool(max(max_consec) <= 7),
        "shift_cover_ok": shift_cover,
        "max_consec_distribution": {f"{k}": int(sum(1 for x in max_consec
if x == k)) for k in sorted(set(max_consec))},
    }

def main() -> None:
    q2 = json.loads((RESULTS_DIR /
"q2_solution.json").read_text(encoding="utf-8"))
    D = np.array(q2["Z_per_day"], dtype=int)
    n_per_shift = q2["n_per_shift_per_day"]
    r = solve(D, time_limit=180)
    w_arr = r["w"]
    a_arr = stage2_assign_to_shifts(w_arr, n_per_shift)
    vstat = verify(w_arr, a_arr, D, n_per_shift)
    print("Verify:", vstat, flush=True)
    out = {
        "N": int(r["N"]),
        "N_LB": int(r["N_LB"]),
        "stage1_status": r["status"],
        "stage1_elapsed_s": r["elapsed_s"],
        "n_patterns_total": r["n_patterns_total"],
        "n_patterns_used": r["n_patterns_used"],
        "used_patterns": r["used_patterns"],
        "D_per_day": D.tolist(),
        "w_per_worker_per_day": w_arr.tolist(),
        "a_per_worker_per_day_per_shift": a_arr.tolist(),
        "verify": vstat,
    }
    (RESULTS_DIR / "q3_solution.json").write_text(
        json.dumps(out, ensure_ascii=False, indent=2), encoding="utf-8")
    print("Saved q3_solution.json", flush=True)

if __name__ == "__main__":
    main()

```

9.5.6 code/fig_q1.py

```

"""T5: Q1 数据图."""

from __future__ import annotations
import json
import numpy as np

```

```

import matplotlib.pyplot as plt

from utils import (C, FIGURES_DIR, RESULTS_DIR, N_DAYS, N_HOURS,
                   build_A, load_volume, setup_chinese_font, save_pdf)

def main() -> None:
    setup_chinese_font()
    v = load_volume()
    sol = json.loads((RESULTS_DIR /
"q1_solution.json").read_text(encoding="utf-8"))
    xi = np.array(sol["xi_per_day_per_start"])
    Z_d = np.array(sol["Z_per_day"])
    starts_per_day = sol["starts_per_day"]
    lb = np.array(sol["lower_bound_per_day"])

    days = np.arange(1, 31)

    # F-Q1-1: 每日总工人数 + 下界
    fig, ax = plt.subplots(figsize=(9, 4.5))
    ax.bar(days, Z_d, color="C0", alpha=0.85, label="Q1 每日工人数 $Z_d$")
    ax.plot(days, lb, "C3o-", label="下界 $L_d$", markersize=5)
    ax.set_xlabel("日序")
    ax.set_ylabel("工人数")
    ax.set_xticks(days[::2])
    ax.grid(alpha=0.3, axis="y")
    ax.legend()
    save_pdf(fig, "F-Q1-1_daily_workers.pdf")
    plt.close(fig)

    # F-Q1-2: 3 个典型日 (min/median/max Z) 小时人力 vs 需求
    A = build_A()
    per_hour = xi @ A # (30, 24)
    cap = per_hour * C
    Z_sorted = np.argsort(Z_d)
    typical = [("min", Z_sorted[0]), ("中位",
Z_sorted[len(Z_sorted)//2]), ("max", Z_sorted[-1])]
    fig, axes = plt.subplots(3, 1, figsize=(9, 8.5), sharex=True)
    hours = np.arange(24)
    for ax, (label, d) in zip(axes, typical):
        ax.bar(hours, v[d], color="C0", alpha=0.5, label="进货量 $v_h$")
        ax.plot(hours, cap[d], "C3-o", label="有效产能 $25 w_h$")
        ax.set_ylabel(f"d={d+1} ({label}), $Z={Z_d[d]}$")

```

```

        ax.grid(alpha=0.3)
        ax.legend(loc="upper left", fontsize=8)
        axes[-1].set_xlabel("小时")
        axes[-1].set_xticks(hours)
        save_pdf(fig, "F-Q1-2_typical_hourly.pdf")
        plt.close(fig)

# F-Q1-3: 起点选择热度
starts_count = np.zeros(N_HOURS, dtype=int)
for sd in starts_per_day:
    for s in sd:
        starts_count[s] += 1
fig, ax = plt.subplots(figsize=(9, 3.5))
ax.bar(np.arange(N_HOURS), starts_count, color="C2", alpha=0.85)
ax.set_xlabel("班次起点小时")
ax.set_ylabel("被选用天数")
ax.set_xticks(np.arange(N_HOURS))
ax.grid(alpha=0.3, axis="y")
save_pdf(fig, "F-Q1-3_start_frequency.pdf")
plt.close(fig)

print("Q1 figures done.")

if __name__ == "__main__":
    main()

```

9.5.7 code/fig_q2.py

```

"""T6: Q2 数据图."""

from __future__ import annotations
import json
import numpy as np
import matplotlib.pyplot as plt

from utils import (C, C_LOW, FIGURES_DIR, RESULTS_DIR, N_DAYS, N_HOURS,
                   build_A, load_volume, setup_chinese_font, save_pdf)

def main() -> None:
    setup_chinese_font()
    v = load_volume()
    q2 = json.loads((RESULTS_DIR /

```

```

"q2_solution.json").read_text(encoding="utf-8"))
    q1 = json.loads((RESULTS_DIR /
"q1_solution.json").read_text(encoding="utf-8"))
    xi = np.array(q2["xi_per_day_per_start"])
    y = np.array(q2["y_per_day_per_start_per_hour"])
    Z2 = np.array(q2["Z_per_day"])
    Z1 = np.array(q1["Z_per_day"])
    days = np.arange(1, 31)

    A = build_A()
    W = xi @ A
    Y = y.sum(axis=1)
    cap = C * W - (C - C_LOW) * Y
    cumcap = cap.cumsum(axis=1)
    cumv = v.cumsum(axis=1)

    # F-Q2-1: 累计进货 vs 累计产能 + 16 点垂线 (3 个典型日)
    Z_sorted = np.argsort(Z2)
    typical = [("min", Z_sorted[0]), ("中位",
Z_sorted[len(Z_sorted)//2]), ("max", Z_sorted[-1])]
    fig, axes = plt.subplots(3, 1, figsize=(9, 8.5), sharex=True)
    hours = np.arange(24)
    for ax, (label, d) in zip(axes, typical):
        ax.plot(hours, cumv[d], "C0-o", label="累计进货  $\sum v$ ")
        ax.plot(hours, cumcap[d], "C3-s", label="累计产能  $\sum$ 
25W-15Y")
        ax.axvline(15.5, color="gray", ls=":", label="16:00 截止")
        ax.set_ylabel(f"d={d+1} ({label}),  $Z={Z2[d]}$ ")
        ax.grid(alpha=0.3)
        ax.legend(loc="upper left", fontsize=8)
    axes[-1].set_xlabel("小时")
    axes[-1].set_xticks(hours)
    save_pdf(fig, "F-Q2-1_cumulative_deadline.pdf")
    plt.close(fig)

    # F-Q2-2: Q2 vs Q1 每日人数对比
    fig, ax = plt.subplots(figsize=(9, 4.5))
    width = 0.4
    ax.bar(days - width/2, Z1, width, color="C0", alpha=0.85, label="Q1
 $Z^{(1)}$ ")
    ax.bar(days + width/2, Z2, width, color="C3", alpha=0.85, label="Q2
 $Z^{(2)}$ ")
    ax.set_xlabel("日序")

```

```

ax.set_ylabel("工人数")
ax.set_xticks(days[::2])
ax.grid(alpha=0.3, axis="y")
ax.legend()
save_pdf(fig, "F-Q2-2_Q2_vs_Q1.pdf")
plt.close(fig)

# F-Q2-3: 低产能小时分布 (按起点) - 显示 y 在 24 小时的分布
fig, ax = plt.subplots(figsize=(9, 3.5))
total_low = Y.sum(axis=0) # (24,) sum over days
ax.bar(np.arange(N_HOURS), total_low, color="C4", alpha=0.85)
ax.set_xlabel("小时")
ax.set_ylabel("低产能工人 (累计 30 日)")
ax.set_xticks(np.arange(N_HOURS))
ax.grid(alpha=0.3, axis="y")
save_pdf(fig, "F-Q2-3_low_cap_hour_distribution.pdf")
plt.close(fig)
print("Q2 figures done.")

if __name__ == "__main__":
    main()

```

9.5.8 code/fig_q3.py

```

"""T7: Q3 数据图."""

from __future__ import annotations
import json
import numpy as np
import matplotlib.pyplot as plt

from utils import FIGURES_DIR, RESULTS_DIR, N_DAYS, setup_chinese_font,
save_pdf

def main() -> None:
    setup_chinese_font()
    q3 = json.loads((RESULTS_DIR /
"q3_solution.json").read_text(encoding="utf-8"))
    w = np.array(q3["w_per_worker_per_day"], dtype=int) # (N, 30)
    D = np.array(q3["D_per_day"])
    N = w.shape[0]
    days = np.arange(1, 31)

```

```

# F-Q3-1: 工人-日 出勤矩阵 (按出勤日期模式排序后)
# 排序: 按 pattern hash
keys = [tuple(w[i]) for i in range(N)]
order = sorted(range(N), key=lambda i: keys[i])
w_sorted = w[order]

fig, ax = plt.subplots(figsize=(9, 6))
ax.imshow(w_sorted, aspect="auto", cmap="Greys",
interpolation="nearest")
ax.set_xlabel("日序")
ax.set_ylabel("工人编号 (按出勤模式排序)")
ax.set_xticks(np.arange(30)[::2])
ax.set_xticklabels(days[::2])
save_pdf(fig, "F-Q3-1_worker_calendar.pdf")
plt.close(fig)

# F-Q3-2: 每日出勤总数 vs 需求 D_d
daily_total = w.sum(axis=0)
fig, ax = plt.subplots(figsize=(9, 4.5))
width = 0.4
ax.bar(days - width/2, D, width, color="C0", alpha=0.85, label="$D_d$
(Q2 需求)")
ax.bar(days + width/2, daily_total, width, color="C2", alpha=0.85,
label="出勤总数 $\sum_i w_{i,d}$")
ax.set_xlabel("日序")
ax.set_ylabel("人数")
ax.set_xticks(days[::2])
ax.grid(alpha=0.3, axis="y")
ax.legend()
save_pdf(fig, "F-Q3-2_daily_coverage.pdf")
plt.close(fig)

# F-Q3-3: 每人最长连续工作天数分布
max_consec = []
for i in range(N):
    run = 0; m = 0
    for d in range(N_DAYS):
        run = run + 1 if w[i, d] else 0
        if run > m: m = run
    max_consec.append(m)
fig, ax = plt.subplots(figsize=(7, 4))
vals, counts = np.unique(max_consec, return_counts=True)

```

```

ax.bar(vals, counts, color="C5", alpha=0.85)
for vi, ci in zip(vals, counts):
    ax.text(vi, ci + N*0.005, str(ci), ha="center")
ax.set_xlabel("最长连续工作天数")
ax.set_ylabel("工人数")
ax.set_xticks(vals)
ax.grid(alpha=0.3, axis="y")
save_pdf(fig, "F-Q3-3_max_consec_distribution.pdf")
plt.close(fig)
print("Q3 figures done.")

if __name__ == "__main__":
    main()

```

9.5.9 code/fig_drawio_pdf.py

"""把 4 张非数据图 (roadmap / flow_q1 / flow_q2 / flow_q3 / model) 渲染为论文 PDF。

不依赖 drawio CLI, 使用 matplotlib 直接绘制等价版本, 保证论文可引用。

"""

```

from __future__ import annotations
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch, FancyArrowPatch
from utils import FIGURES_DIR, setup_chinese_font

```

```

setup_chinese_font()

```

```

NODE_STYLE_PROC = dict(boxstyle="round,pad=0.4", fc="#fff2cc",
ec="#d6b656", lw=1.4)
NODE_STYLE_DATA = dict(boxstyle="round,pad=0.4", fc="#d5e8d4",
ec="#82b366", lw=1.4)
NODE_STYLE_DEC = dict(boxstyle="round,pad=0.3", fc="#f8cecc",
ec="#b85450", lw=1.4)
NODE_STYLE_END = dict(boxstyle="round,pad=0.4", fc="#e1d5e7",
ec="#9673a6", lw=1.4)
NODE_STYLE_START = dict(boxstyle="round,pad=0.4", fc="#dae8fc",
ec="#6c8ebf", lw=1.4)

```

```

def node(ax, x, y, w, h, text, style, fontsize=10):
    ax.text(x, y, text, ha="center", va="center", fontsize=fontsize,

```

```

        bbox=style, wrap=True)

def arrow(ax, x1, y1, x2, y2, label=""):
    ax.annotate("", xy=(x2, y2), xytext=(x1, y1),
                arrowprops=dict(arrowstyle="->", color="#333", lw=1.3))
    if label:
        ax.text((x1 + x2) / 2 + 0.2, (y1 + y2) / 2, label, fontsize=9,
                color="#555")

```

9.5.10 code/fig_roadmap.py

```

"""技术路线图 PDF。"""
import matplotlib.pyplot as plt
from utils import FIGURES_DIR, setup_chinese_font
from fig_drawio_pdf import (NODE_STYLE_PROC, NODE_STYLE_DATA,
                             NODE_STYLE_END,
                             NODE_STYLE_START, node, arrow)

setup_chinese_font()
fig, ax = plt.subplots(figsize=(11, 7.5))
ax.set_xlim(0, 11); ax.set_ylim(0, 7.5); ax.axis("off")
ax.text(5.5, 7.1, "物流分拣中心排班—技术路线", ha="center", fontsize=15,
        fontweight="bold")

# left column
node(ax, 1.4, 6.0, 0, 0, "附件 1\nv$_{d,h}$ (30×24)", NODE_STYLE_START)
node(ax, 1.4, 4.7, 0, 0, "EDA / 节律 / 下界 L$_d$ \n日间相关 0.93",
    NODE_STYLE_DATA)
node(ax, 1.4, 3.2, 0, 0, "假设决断 G1-G5\n整点起点 / 累计口径\n低产能由排班指
派", NODE_STYLE_PROC, fontsize=9)

# middle column
node(ax, 5.5, 6.0, 0, 0, "问题一\n每日 MILP (枚举 C(24,5)) \nmin  $\sum s_s$ ",
    NODE_STYLE_DATA, fontsize=10)
node(ax, 5.5, 4.4, 0, 0, "问题二\nQ1 + 16 点截止\n+ 班内低产能 y$_{s,h}$",
    NODE_STYLE_DATA, fontsize=10)
node(ax, 5.5, 2.8, 0, 0, "问题三\n模式生成 + LP + MILP\nmin N s.t. 23 天 /
≤7 连工", NODE_STYLE_DATA, fontsize=10)

# right column
node(ax, 9.4, 4.4, 0, 0, "结果\nZ$^{(1)}$=10791 Z$^{(2)}$=11668\nN=581
(均达下界)", NODE_STYLE_END, fontsize=10)
node(ax, 9.4, 2.8, 0, 0, "校验\n下界对比 / 单调性 / 约束回代",

```

```

NODE_STYLE_DATA, fontsize=9)
node(ax, 9.4, 1.2, 0, 0, "Typst 论文撰写", NODE_STYLE_PROC, fontsize=10)

# arrows
arrow(ax, 1.4, 5.7, 1.4, 5.0)
arrow(ax, 1.4, 4.4, 1.4, 3.6)
arrow(ax, 2.5, 3.2, 4.5, 5.8)
arrow(ax, 5.5, 5.5, 5.5, 4.9)
arrow(ax, 5.5, 4.0, 5.5, 3.3)
arrow(ax, 6.6, 4.4, 8.5, 4.4)
arrow(ax, 9.4, 4.0, 9.4, 3.3)
arrow(ax, 9.4, 2.4, 9.4, 1.6)

fig.savefig(FIGURES_DIR / "fig_roadmap.pdf", format="pdf",
bbox_inches="tight")
plt.close(fig)
print("fig_roadmap.pdf done")

```

9.5.11 code/fig_flow_q1_pdf.py

```

"""问题一求解流程 PDF。"""
import matplotlib.pyplot as plt
from utils import FIGURES_DIR, setup_chinese_font
from fig_drawio_pdf import (NODE_STYLE_PROC, NODE_STYLE_DATA,
                             NODE_STYLE_DEC,
                             NODE_STYLE_END, NODE_STYLE_START, node,
                             arrow)

setup_chinese_font()
fig, ax = plt.subplots(figsize=(8, 11))
ax.set_xlim(0, 8); ax.set_ylim(0, 11); ax.axis("off")
ax.text(4, 10.6, "问题一求解流程: 每日独立 + 双级目标", ha="center",
        fontsize=13, fontweight="bold")

steps = [
    (4, 10.0, "开始", NODE_STYLE_START),
    (4, 9.2, "读取  $v_{d,h}$ , 构造覆盖矩阵  $A_{s,h}$  ( $24 \times 24$ ) ",
NODE_STYLE_DATA),
    (4, 8.3, "对每日 d: 计算累计  $\text{cumv}_H$  与下界  $L_d$ ", NODE_STYLE_DATA),
    (4, 7.4, "枚举  $C(24,5)=42504$  个起点组合 s", NODE_STYLE_PROC),
    (4, 6.5, "对每个 s 解 5 变量 LP  $\min \sum x_k$  s.t.  $C \cdot M_s \cdot x \geq \text{cumv}$ ",
NODE_STYLE_PROC),
    (4, 5.4, " $Z_{LP} <$  当前最优?", NODE_STYLE_DEC),
    (4, 4.3, "向上取整 + 补救  $\xi$  计算  $\text{peak} = \max_h W_h$  按 (Z, peak) 字

```

```

典序更新最优", NODE_STYLE_END),
    (4, 3.0, "所有组合枚举完毕?", NODE_STYLE_DEC),
    (4, 1.9, "保存 {${d}, ·} ${starts${d}}; d←d+1", NODE_STYLE_DATA),
    (4, 1.0, "所有日已处理?", NODE_STYLE_DEC),
    (4, 0.2, "导出 q1_solution.json + 校验 ≥ L", NODE_STYLE_END),
]
for x, y, text, style in steps:
    node(ax, x, y, 0, 0, text, style, fontsize=9)

for i in range(len(steps) - 1):
    arrow(ax, steps[i][0], steps[i][1] - 0.25, steps[i+1][0], steps[i+1][1] + 0.25)

# 是/否 labels
ax.text(4.1, 5.0, "是", fontsize=9, color="#555")
ax.text(5.5, 5.3, "否", fontsize=9, color="#555")
# back-loop arrows
ax.annotate("", xy=(6.5, 6.5), xytext=(6.5, 3.0),
            arrowprops=dict(arrowstyle="→", color="#999", lw=1.0,
connectionstyle="arc3,rad=0.2"))
ax.text(6.7, 4.7, "否", fontsize=9, color="#555")
ax.annotate("", xy=(0.6, 8.3), xytext=(0.6, 1.0),
            arrowprops=dict(arrowstyle="→", color="#999", lw=1.0))
ax.text(0.3, 4.7, "否", fontsize=9, color="#555")

fig.savefig(FIGURES_DIR / "fig_flow_q1.pdf", format="pdf",
bbox_inches="tight")
plt.close(fig)
print("fig_flow_q1.pdf done")

```

9.5.12 code/fig_flow_q2_pdf.py

```

"""问题二求解流程 PDF。"""
import matplotlib.pyplot as plt
from utils import FIGURES_DIR, setup_chinese_font
from fig_drawio_pdf import (NODE_STYLE_PROC, NODE_STYLE_DATA,
NODE_STYLE_DEC,
                                NODE_STYLE_END, NODE_STYLE_START, node,
arrow)

setup_chinese_font()
fig, ax = plt.subplots(figsize=(8, 10))
ax.set_xlim(0, 8); ax.set_ylim(0, 10); ax.axis("off")
ax.text(4, 9.6, "问题二求解流程: MILP + 班内低产能小时", ha="center",

```

```

fontsize=13, fontweight="bold")

steps = [
    (4, 9.0, "开始", NODE_STYLE_START),
    (4, 8.2, "读取  $v_{d,h}$  与 Shift(s) (模 24)", NODE_STYLE_DATA),
    (4, 7.4, "对每日 d: 构造 PuLP MILP", NODE_STYLE_DATA),
    (4, 6.4, "决策变量  $\delta_{s,h} \in \{0,1\}$ ;  $y_{s,h} \geq 0$  连续  
( $h \in \text{Shift}(s)$ )", NODE_STYLE_PROC),
    (4, 5.0, "约束  $\sum \delta = 5$ ; 班内  $\sum y_{s,h} = \xi_{s,h}$  累计  $\text{Cap}_h \geq$  累  
计  $v_h$  (每个 H)", NODE_STYLE_PROC),
    (4, 3.7, " $\text{Cap}_h = 25 \cdot \sum A_{s,h} - 15 \cdot \sum y_{s,h}$   
 $y_{s,h}$ ", NODE_STYLE_END),
    (4, 2.6, "CBC 求解 (threads=2, timeLimit=30s)", NODE_STYLE_DEC),
    (4, 1.6, "校验  $\text{CumCap}_{15} \geq \text{CumArr}_{11}$  (D3)  $\text{Z}^{(2)}_d \geq$   
 $\text{Z}^{(1)}_d$  单调性", NODE_STYLE_DATA),
    (4, 0.3, "导出 q2_solution.json", NODE_STYLE_END),
]
for x, y, text, style in steps:
    node(ax, x, y, 0, 0, text, style, fontsize=9)

for i in range(len(steps) - 1):
    arrow(ax, steps[i][0], steps[i][1] - 0.27, steps[i+1][0], steps[i+1][1] + 0.27)

fig.savefig(FIGURES_DIR / "fig_flow_q2.pdf", format="pdf",
bbox_inches="tight")
plt.close(fig)
print("fig_flow_q2.pdf done")

```

9.5.13 code/fig_flow_q3_pdf.py

```

"""问题三求解流程 PDF。"""
import matplotlib.pyplot as plt
from utils import FIGURES_DIR, setup_chinese_font
from fig_drawio_pdf import (NODE_STYLE_PROC, NODE_STYLE_DATA,
                             NODE_STYLE_DEC,
                             NODE_STYLE_END, NODE_STYLE_START, node,
                             arrow)

setup_chinese_font()
fig, ax = plt.subplots(figsize=(9, 10))
ax.set_xlim(0, 9); ax.set_ylim(0, 10); ax.axis("off")
ax.text(4.5, 9.6, "问题三求解流程: 模式 + LP 松弛 + MILP 精化", ha="center",
        fontsize=13, fontweight="bold")

```

```

steps = [
    (4.5, 9.0, "开始", NODE_STYLE_START, ""),
    (4.5, 8.2, "读取 Q2 输出 D$_d$ 与 n$_{d,k}$", NODE_STYLE_DATA, ""),
    (4.5, 7.4, "下界 N$_{LB}$ = max(ceil( $\sum D/23$ ), max D$_d$) = 581",
NODE_STYLE_DATA, ""),
    (4.5, 6.5, "枚举合法 30 日模式\nsum=23, 最长连续 1  $\leq$  7", NODE_STYLE_PROC,
"767,544 条"),
    (4.5, 5.4, "LP 松弛 (scipy.linprog HiGHS)\nmin  $\sum n\_p$ s.t. P$^T n \geq$ 
D,  $n \geq 0$ ", NODE_STYLE_PROC, "LP 最优 = 581.00"),
    (4.5, 4.3, "筛 LP 活跃模式 (n$_p$ > 0)", NODE_STYLE_END, "14 条活跃"),
    (4.5, 3.4, "MILP 精化 (CBC)\nn$_p$  $\in$  z$_+$ 限定在活跃模式",
NODE_STYLE_DEC, ""),
    (4.5, 2.4, "N =  $\sum n\_p$; 展开 w$_{i,d}$ (N $\times$ 30)", NODE_STYLE_DATA, ""),
    (4.5, 1.6, "第二阶段: 每日 D$_d$ 工人贪心分到 5 班次", NODE_STYLE_DATA,
""),
    (4.5, 0.8, "校验: 23 天 /  $\leq$  7 连工 / 班次覆盖", NODE_STYLE_DATA, ""),
    (4.5, 0.05, "导出 q3_solution.json (N=581)", NODE_STYLE_END, ""),
]

for x, y, text, style, _ in steps:
    node(ax, x, y, 0, 0, text, style, fontsize=9)

# annotations
for x, y, _, _, anno in steps:
    if anno:
        ax.text(7.2, y, anno, fontsize=9, color="#666", style="italic")

for i in range(len(steps) - 1):
    arrow(ax, steps[i][0], steps[i][1] - 0.25, steps[i+1][0], steps[i+1][1] + 0.25)

fig.savefig(FIGURES_DIR / "fig_flow_q3.pdf", format="pdf",
bbox_inches="tight")
plt.close(fig)
print("fig_flow_q3.pdf done")$ 
```

9.5.14 code/fig_model_pdf.py

```

"""模型变量关系图 PDF。"""
import matplotlib.pyplot as plt
from utils import FIGURES_DIR, setup_chinese_font
from fig_drawio_pdf import (NODE_STYLE_PROC, NODE_STYLE_DATA,
NODE_STYLE_DEC,
NODE_STYLE_END, NODE_STYLE_START, node,

```

```

arrow)

setup_chinese_font()
fig, ax = plt.subplots(figsize=(11.5, 7))
ax.set_xlim(0, 11.5); ax.set_ylim(0, 7); ax.axis("off")
ax.text(5.75, 6.6, "模型变量与约束关系", ha="center", fontsize=14,
fontweight="bold")

# Inputs (left)
node(ax, 1.0, 4.0, 0, 0, "输入\n $v_{d,h}$ ", NODE_STYLE_START, 11)
# Decisions (middle-left)
node(ax, 3.0, 5.7, 0, 0, " $\delta_s \in \{0,1\}$ \n起点是否选用", NODE_STYLE_PROC,
10)
node(ax, 3.0, 4.0, 0, 0, " $\xi_{d,s} \in \mathbb{Z}_+$ \n每日每起点工人数",
NODE_STYLE_PROC, 10)
node(ax, 3.0, 2.3, 0, 0, " $y_{d,s,h} \geq 0$ \n班内低产能小时",
NODE_STYLE_PROC, 10)
# Derived (middle)
node(ax, 5.7, 4.9, 0, 0, " $W_{d,h} = \sum_s A_{s,h} \xi_{d,s}$ \n小时在岗",
NODE_STYLE_DATA, 10)
node(ax, 5.7, 3.0, 0, 0, " $Cap_{d,h} = 25 \cdot W_{d,h} - 15 \cdot \sum_s y_{d,s,h}$ ",
NODE_STYLE_DATA, 10)
# Constraints (right)
node(ax, 9.0, 5.9, 0, 0, " $\sum_s \delta_s = 5$  (恰 5 起点)", NODE_STYLE_DEC,
10)
node(ax, 9.0, 5.0, 0, 0, " $\xi_{d,s} \leq M \cdot \delta_s$ ", NODE_STYLE_DEC, 10)
node(ax, 9.0, 3.5, 0, 0, " $\sum_{h \in \text{Shift}(s)} y_{d,s,h} = \xi_{d,s}$ ",
NODE_STYLE_DEC, 10)
node(ax, 9.0, 2.4, 0, 0, " $\sum_{h \leq H} Cap_{d,h} \geq \sum_{h \leq H} v_{d,h}$ \n(每个 H)",
NODE_STYLE_DEC, 10)
node(ax, 9.0, 1.0, 0, 0, "目标\nQ1/Q2:  $\min \sum \xi$  ; Q3:  $\min N$ ",
NODE_STYLE_END, 10)

# arrows
arrow(ax, 1.5, 4.0, 2.4, 4.0)
arrow(ax, 3.0, 5.4, 3.0, 4.4)
arrow(ax, 3.6, 4.0, 5.1, 4.7)
arrow(ax, 3.6, 4.0, 5.1, 3.2)
arrow(ax, 3.6, 2.3, 5.1, 2.7)
arrow(ax, 1.5, 4.0, 5.0, 3.0)
arrow(ax, 6.4, 3.0, 8.4, 2.4)
arrow(ax, 3.6, 5.7, 8.4, 5.9)
arrow(ax, 3.6, 5.7, 8.4, 5.0)

```

```
arrow(ax, 3.6, 2.3, 8.4, 3.5)

fig.savefig(FIGURES_DIR / "fig_model.pdf", format="pdf",
bbox_inches="tight")
plt.close(fig)
print("fig_model.pdf done")
```